
HSBI: IFM 3.2: Grundlagen der KI (Winter 2026/27)

Canan Yıldız (TDU) & Carsten Gips (HSBI)

4. September 2026

Inhaltsverzeichnis

I. Syllabus	1
1. Syllabus HSBI	2
1.1. Kursbeschreibung	2
1.2. Überblick Modulinhalt	2
1.3. Team	3
1.4. Kursformat (HSBI)	3
1.5. Fahrplan (HSBI)	3
1.6. Prüfungsform, Note und Credits (HSBI)	4
1.7. Materialien	5
II. Vorlesungsunterlagen	6
2. Einführung KI	7
3. Suche	8
3.1. Search: Lokale Suche - Gradientensuche	8
3.1.1. Unterschiede in den Suchproblemen?	8
3.1.2. Analogie: Bergsteigen ohne Karte und Pfade	9
3.1.3. Pseudoalgorithmus Gradientensuche	10
3.1.4. Beispiel Gradientensuche: n -Damen	10
3.1.5. Eigenschaften Gradientensuche	11
3.1.6. Wrap-Up	12
3.2. Search: Lokale Suche - Simulated Annealing	13
3.2.1. Motivation	13
3.2.2. Gedankenexperiment: Ausweg aus lokalen Minima	14
3.2.3. Analogie Härten von Metall	14
3.2.4. Übertragen der Idee	14
3.2.5. Pseudocode Simulated Annealing (Minimierungsproblem)	14
3.2.6. Detail: Akzeptieren von Verschlechterungen	15
3.2.7. Abkühlungsplan problemabhängig wählen	15
3.2.8. Eigenschaften Simulated Annealing	16
3.2.9. Anwendungen von Simulated Annealing	16
3.2.10. Wrap-Up	16
4. Genetische Algorithmen	18

5. Spiele	19
6. Entscheidungsbäume (Decision Tree Learner - DTL)	20
6.1. DTL: Machine Learning 101	20
6.1.1. Was ist Lernen?	21
6.1.2. Warum Lernen?	21
6.1.3. Learning Agent	21
6.1.4. Feedback während des Lernens	21
6.1.5. Beispiel: Kreditrisiko	22
6.1.6. Beispiel: Autoreparatur	22
6.1.7. Lernen durch Beobachten: Lernen einer Funktion f	23
6.1.8. Konstruieren einer konsistenten Hypothese	23
6.1.9. Konstruieren einer konsistenten Hypothese (cnt.)	24
6.1.10. Konstruieren einer konsistenten Hypothese (cnt.)	24
6.1.11. Konstruieren einer konsistenten Hypothese (cnt.)	25
6.1.12. Konstruieren einer konsistenten Hypothese (cnt.)	25
6.1.13. Occam's Razor	26
6.1.14. Trainingsdaten und Merkmalsvektoren	26
6.1.15. Beispiel: Schwimmen im See	26
6.1.16. Trainingsdaten – Merkmalsvektoren	27
6.1.17. Wrap-Up	27
7. Constraintsolving	29
8. NN: Einführung in Neuronale Netze	30
8.1. NN01: Das Perzeptron	30
8.1.1. Kurze Übersicht	30
8.2. NN02: Lineare Regression und Gradientenabstieg	34
8.2.1. Kurze Übersicht	34
8.3. NN03: Logistische Regression	37
8.3.1. Kurze Übersicht	38
8.4. NN04: Overfitting und Regularisierung	40
8.4.1. Kurze Übersicht	40
8.5. NN05: Multilayer Perzeptron	41
8.5.1. Kurze Übersicht	41
8.6. NN06: Backpropagation	43
8.6.1. Kurze Übersicht	43
8.7. NN07: Training & Testing	44
8.7.1. Kurze Übersicht	45
8.8. NN08: Performanzanalyse	50
8.8.1. Kurze Übersicht	50
9. Naive Bayes	53

III. Organisatorisches	54
10. Prüfungsvorbereitung (HSBI)	55
10.1. Elektronische Klausur: Termin, Materialien	55
10.1.1. Termin	55
10.1.2. Zugelassene Hilfsmittel: DIN-A4-Spickzettel (beidseitig)	56
10.1.3. Einsicht	56
10.2. Technische Vorbereitungen	56
10.3. Bearbeitung des E-Assessment	56
10.4. Fragetypen-Demo	57
10.5. Hinweise zu den Inhalten	57
10.6. Beispiele für mögliche Fragen	57
10.6.1. Breitensuche	57
10.6.2. Suche und Constraints	58
10.6.3. Handsimulation Perzeptron	58
10.6.4. Entscheidungsbäume mit ID3	58
10.6.5. CSP	59
10.6.6. NB	59

Teil I.

Syllabus

1. Syllabus HSBI



Quelle: "künstliche Intelligenz" by Gerd Altmann (geralt) on Pixabay.com (Pixabay License)

1.1. Kursbeschreibung

Ausgehend von den Fragen "Was ist *Intelligenz*?" und "Was ist *künstliche* Intelligenz?" werden wir uns in diesem Modul mit **verschiedenen Teilgebieten der KI** beschäftigen und uns anschauen, welche **Methoden und Algorithmen** es gibt und wie diese funktionieren. Dabei werden wir auch das Gebiet *Machine Learning* berühren, aber auch andere wichtige Gebiete betrachten. Sie erarbeiten sich im Laufe der Veranstaltung einen **Methoden-Baukasten** zur Lösung unterschiedlichster Probleme und erwerben ein grundlegendes Verständnis für die Anwendung in Spielen, Navigation, Planung, smarten Assistenten, autonomen Fahrzeugen, ...

1.2. Überblick Modulinhalte

1. Problemlösen

- Zustände, Aktionen, Problemraum
- Suche (blind, informiert): Breiten-, Tiefensuche, Best-First, Branch-and-Bound, A-Stern
- Lokale Suche: Gradientenabstieg, Genetische/Evolutionäre Algorithmen (GA/EA)
- Spiele: Minimax, Alpha-Beta-Pruning, Heuristiken
- Constraints: Backtracking, Heuristiken, Propagation, AC-3

2. Maschinelles Lernen

- Merkmalsvektor, Trainingsmenge, Trainingsfehler, Generalisierung
- Entscheidungsbäume: CAL2, CAL3, ID3, C4.5
- Neuronale Netze
 - Perzeptron, Lernregel

- Feedforward Multilayer Perzeptron (MLP), Backpropagation, Trainings- vs. Generalisierungsfehler
- Steuerung des Trainings: Kreuzvalidierung, Regularisierung
- Ausblick: Support-Vektor-Maschinen
- Naive Bayes Klassifikator

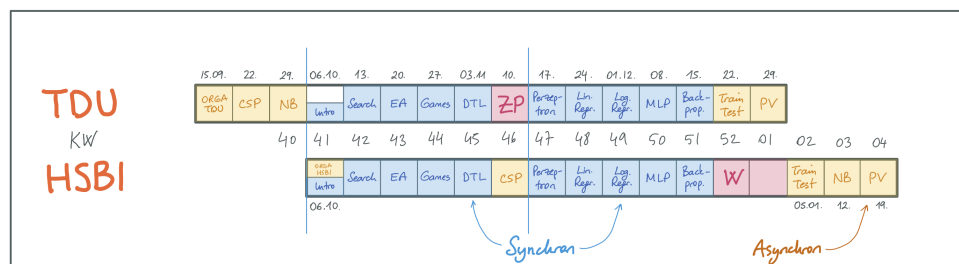
3. Inferenz, Logik (**entfällt im W26**)

- **Prädikatenlogik: Modellierung, semantische und formale Beweise, Unifikation, Resolution**
- **Ausblick: Anwendung in Prolog**

1.3. Team

- **Carsten Gips** (HSBI, Sprechstunde nach Vereinbarung)
- **Canan Yıldız** (TDU)

1.4. Kursformat (HSBI)



Vorlesung (2 SWS)

Praktikum (2 SWS)

Mo, 09:00 - 10:30 Uhr (DE)

G1: Mo, 10:45 - 12:15 Uhr (DE)

(Flipped Classroom)

G2: Mi, 15:45 - 17:15 Uhr (DE)

G3: Mo, 14:00 - 15:30 Uhr (DE)

G4: Do, 14:00 - 15:30 Uhr (DE)

Alle Sitzungen online per Zoom (**Zugangsdaten siehe ILIAS**).

1.5. Fahrplan (HSBI)

Abgabe der Übungsblätter jeweils **Montag bis 09:00 Uhr** im **ILIAS**. Vorstellung der Lösung im jeweiligen Praktikum in der Abgabewoche.

Monat	Woche vom	Vorlesung (Mo)	Praktikum (Mo/Mi/Do)
Oktober	12. ...	Orga Einführung KI & Problemlösen	
	19. ...	Suche (Search)	
	26. ...	Gradientensuche & Simulated Annealing Evolutionäre Algorithmen (EA)	
November	02. ...	Optimale Spiele (Games)	
	09. ...	Entscheidungsbäume (DTL)	
	16. ...	Constraints (CSP)	
	23. ...	Perzeptron	
Dezember	30. ...	Lineare Regression und Gradientenabstieg	
	07. ...	Logistische Regression	
	14. ...	Overfitting und Regularisierung Multilayer Perceptron (MLP)	
	21. ...	Weihnachtspause	
	28. ...	Weihnachtspause	
Januar	04. ...	Backpropagation	
	11. ...	Training & Testing Performanzanalyse	
	18. ...	Naive Bayes (NB)	
	25. ...	Prüfungsvorbereitung HSBI Rückblick	

1.6. Prüfungsform, Note und Credits (HSBI)

(Digitale) Klausur plus Studienleistung (Portfolio), 5 ECTS

- **Studienleistung:** “Portfolio” - Kriterien je Person:
 1. Teilnahme an der Internationalen Projektwoche “AI Connect” mit aktiver Beteiligung (Meeting 1: Team-Building/Kick-Off, Meeting 2: Project Launch, Teamarbeit, Meeting 3: Team-Präsentationen (Vortrag/Video))
 2. Mind. fünf Übungsblätter erfolgreich bearbeitet
 3. Video-Vortrag zum erfolgreich bearbeiteten Mini-Projekt (“AI Connect”) am Fr, 19.12., ab 10:30 Uhr, a 2 Minuten (pro Team)

Je Kriterium: Abgabe eines Post Mortem im ILIAS (**jede Person individuell**)

- **Gesamtnote:** (Digitale) Klausur im B40 (90 Minuten)

Hinweise

- Die Bearbeitung der Leistungen erfolgt individuell (außer “AI Connect” Projektwoche).
- Für die “AI Connect” Projektwoche werden gemischte Teams aus den beteiligten Hochschulen gebildet.
- Die Post Mortems sind individuell zu erstellen und abzugeben.
- “Aktive Beteiligung” umfasst Anwesenheit und sachbezogene Beiträge; Anwesenheit/Beteiligung werden dokumentiert.
- “Erfolgreiche Bearbeitung” eines Blattes umfasst Bearbeitung aller Aufgaben des Blattes und fristgerechte Abgabe des ausreichenden Post Mortems im ILIAS. Die intensive Beschäftigung mit den Aufgaben muss erkennbar sein.
- **Post Mortem:** Jede Person beschreibt individuell(!) die Bearbeitung des jeweiligen Kriteriums bzw. die Teilnahme an den drei Meetings des “AI Connect”-Mini-Projekts zurückblickend mit mind. 150 bis max. 400 Wörtern (Nutzlast; Überschriften und Links zählen nicht mit). Gehen Sie dabei aussagekräftig und nachvollziehbar auf folgende Punkte ein:
 1. Zusammenfassung: Was wurde gemacht bzw. was wurde auf dem Meeting besprochen?
 2. Details: Kurze Beschreibung besonders interessanter Aspekte.
 3. Reflexion: Was war der schwierigste Teil? Wie haben Sie dieses Problem gelöst?
 4. Reflexion: Was haben Sie gelernt oder (besser) verstanden?
 5. Link zu Ihrem Repo mit den relevanten Artefakten (Lösung, Slides für den Vortrag, ...).

Für die “AI Connect”-Meetings passen Sie bitte die Punkte (1) bis (4) entsprechend inhaltlich an, (5) entfällt für die ersten beiden Meetings (für das dritte Meeting (Team-Präsentation) geben Sie bitte einen Link zu Ihrem Repo mit den Artefakten (Lösung, Vortrag, Video) an).

Die Post Mortems geben Sie bitte pro Person bis spätestens zur jeweiligen Deadline im [ILIAS](#) ab.

Siehe auch <https://github.com/Artificial-Intelligence-HSBI-TDU/KI-Vorlesung-W25/discussions/3>.

1.7. Materialien

1. **“Artificial Intelligence: A Modern Approach”** (AIMA). Russell, S. und Norvig, P., Pearson, 2021. ISBN [978-0134610993](#).
2. “Grundkurs Künstliche Intelligenz”. Ertel, W., Springer, 2025. ISBN [978-3-658-44955-1](#). DOI [10.1007/978-3-658-44955-1](#).
3. “An Introduction to Machine Learning”. Kubat, M., Springer, 2017. ISBN [978-3-319-63913-0](#). DOI [10.1007/978-3-319-63913-0](#).

Teil II.

Vorlesungsunterlagen

2. Einführung KI

Was ist Intelligenz? Was ist künstliche Intelligenz? Woran kann man das erkennen? Wie kann man eine Welt (ein Problem) modellieren, um es dann anschließend lösen zu können?

3. Suche

Problemlösen durch Suche im Problemgraphen. Aus den Basisalgorithmen Tree-Search und Graph-Search entstehen je nach verwendeter Datenstruktur und nach betrachteten Kosten unterschiedliche Suchalgorithmen.

- Uninformierte Suche: ... jeder Schritt "kostet" gleich viel: nur die Anzahl der Schritte zählt ...
- Informierte Suche: ... Einsatz einer Kostenfunktion ...
- Lokale Suche: ... das Ziel ist im Weg ...

3.1. Search: Lokale Suche - Gradientensuche

TL;DR

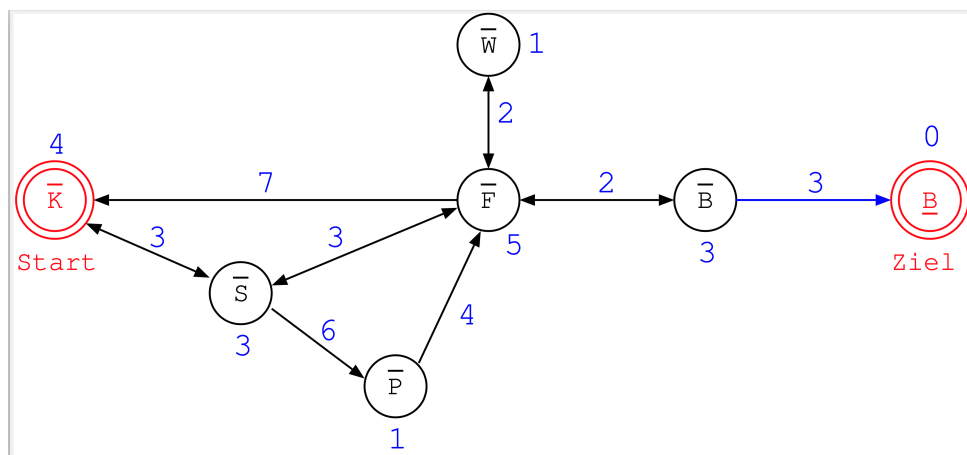
Lokale Suchverfahren: Nur das Ergebnis zählt! Nicht der Weg ist das Ziel, sondern nur das Erreichen des Ziels.

In Analogie zum Bergsteigen: Gehe in Richtung des stärksten Anstiegs kann man die Suche so formulieren, dass man in jedem Suchschritt den Nachfolgeknoten nach dem stärksten Anstieg der Kostenfunktion auswählen. Dieses Verfahren nennt sich auch **Hill-Climbing** (bzw. Gradientensuche).

Videos

- [VL Gradientensuche](#)

3.1.1. Unterschiede in den Suchproblemen?



Montag	Dienstag	Mittwoch	Donnerstag
8:00 9:30 D 319 IFM Mobile Applikationen/V Hoffmann, Martin	8:00 9:30 A 130 IFM Künstl. Intelligenz/V Gips, Carsten	8:00 9:30 A 130 IFM Sicherheit u. Zuverlässigkeit/V Thiel, Christoph	
9:45 13:00 D 319 IFM Mobile Applikationen/P Hoffmann, Martin		9:45 11:15 D 317 IFM Sicherheit u. Zuverlässigkeit/P/Gr. 2,3 Thiel, Christoph	
	11:30 13:00 D 320 IFM Künstl. Intelligenz/P/Gr. 2 Gips, Carsten	11:30 13:00 A 140 IFM Komponente nbasierte SW-Entwicklung/V Brunsmann, Jörg	
		14:00 15:30 C 1 IFM Komponente nbasierte SW-Entwicklung/P/Gr. Brunsmann, Jörg	
15:45 17:15 A 60 IFM Technical English/SU George, Bill			16:30 18:00 A 360 IFM Technical English/SU George, Bill

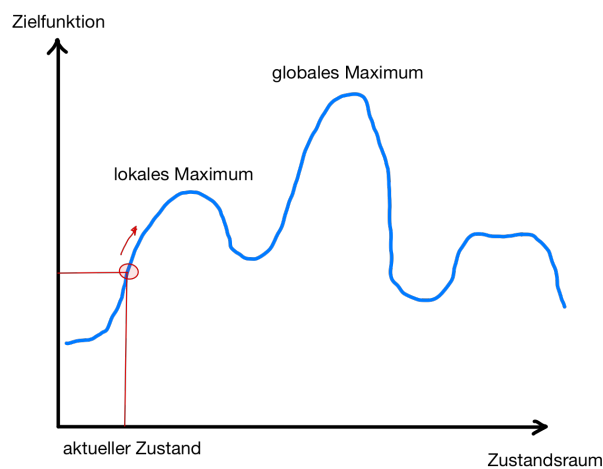
Bisher betrachtete Suchverfahren:

- Systematische Erkundung des Suchraums
- **Weg** zur Lösung wichtig

=> Oft aber nur das **Ziel an sich** interessant!
(Und nicht, wie man dort hin gelangt.)

Beispiel: Stundenplan

3.1.2. Analogie: Bergsteigen ohne Karte und Pfade



Gradienten-Suche: “Gehe in Richtung des steilsten Anstiegs der Zielfunktion.”

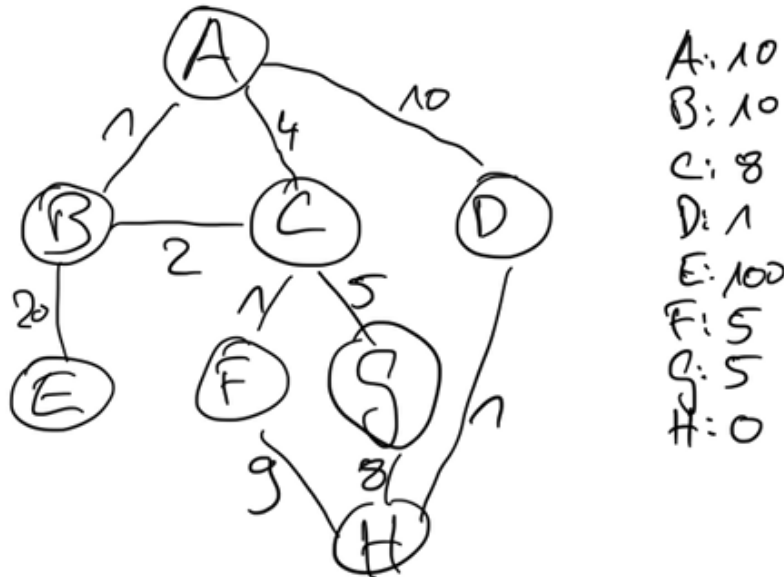
=> Schrittweise Verbesserung des aktuellen Zustands (Lokale Suche)

- Verschiedene Namen: “Hill-climbing”, “Greedy local search”
- Kann auch als Minimierung angewendet werden

3.1.3. Pseudoalgorithmus Gradientensuche

“Wie Bergsteigen am Mount Everest in dickem Nebel mit Gedächtnisverlust”

1. Setze `currNode` auf den Startknoten
2. `currNode` ist gesuchtes Element: Abbruch, melde “gefunden”
 - Expandiere alle Nachfolger von `currNode`
 - Setze `nextNode` auf Nachfolger mit höchster Bewertung
 - Falls Bewertung von `nextNode` $\leq$ Bewertung von `currNode`:
Abbruch, melde “nicht gefunden”
 - Setze `currNode` auf `nextNode`
3. Gehe zu Schritt 2



3.1.4. Beispiel Gradientensuche: n -Damen

- **Ziel:** Setze n Damen auf ein $n \times n$ -Spielfeld ohne Konflikte
- **Start:** Setze n Damen auf ein $n \times n$ -Spielfeld (mit Konflikten)

- **Suche:** Bewege jeweils eine Dame so, daß die Anzahl der Konflikte reduziert wird

Schauen Sie sich auch Abb. 4.3 auf Seite 130 im Russell und Norvig (2021) an!

Hinweis: Alle Damen stehen von Anfang an auf dem Brett und werden nur verschoben => “vollständige Zustandsformulierung”

3.1.4.1. Eigenschaften 8-Damen-Problem ($n = 8$)

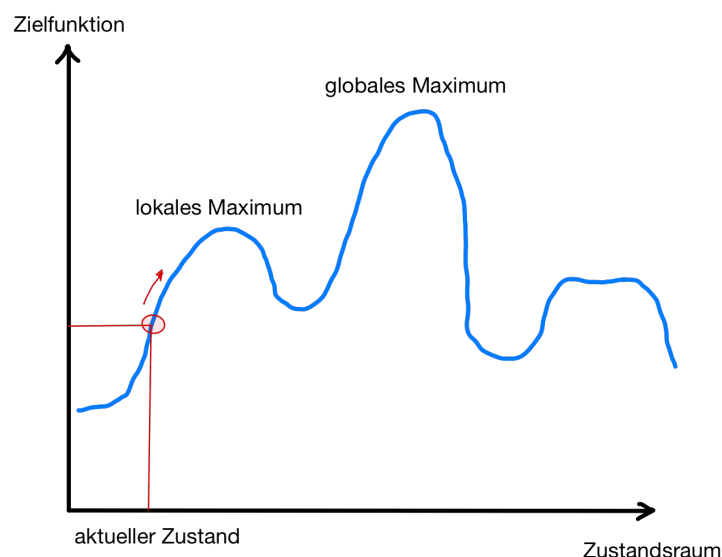
- Zustandsraum: $8^8 \approx 17$ Millionen Zustände!
- Beginnend mit zufällig erzeugtem Startzustand:
 - bleibt in 86% der Fälle stecken, d.h.
 - findet Lösung nur in 14% der Fälle.
- Beobachtung: Lösung nach durchschnittlich 4 Schritten, oder Verfahren bleibt nach durchschnittlich 3 Schritten stecken.

Quelle: nach (Russell und Norvig 2021, p. 131)

3.1.5. Eigenschaften Gradientensuche

- Vollständigkeit: nein
- Optimalität: nein
- Komplexität: linear in der Anzahl der zu expandierenden Knoten

Zielfunktion (Bewertung) nötig!



Problem: lokale Maxima und Plateaus

- Lokale Maxima/Minima: Algorithmus findet nur eine suboptimale Lösung
- Plateaus: Hier muss der Algorithmus mit zufälligen Zügen explorieren

3.1.6. Wrap-Up

Lokale Suchverfahren: Nur das Ergebnis zählt!

- Gradientenverfahren: Gehe in Richtung des stärksten Anstiegs der Kostenfunktion

Zum Nachlesen

- Russell und Norvig (2021): Gradientenabstieg: Abschnitt 4.1.1

Lernziele

- k2: Ich kann die Problematik der lokalen Minima bei Gradientenverfahren erklären
- k3: Ich kann die lokale Suche (Gradientenabstieg) auf ein konkretes Problem anwenden

Quizzes

- [Selbsttest Gradientensuche \(ILIAS\)](#)

Challenges

Betrachten Sie folgende Landkarte und Restwegschätzungen:

Quelle: [MapGermanyGraph.svg](#) by [Regnaror](#) and [Jahobr](#) on Wikimedia Commons ([Public Domain](#))

Stadt	$h(n)$
Augsburg	0 km
Erfurt	400 km
Frankfurt	100 km
Karlsruhe	10 km
Kassel	460 km
Mannheim	200 km
<i>München</i>	0 km
Nürnberg	537 km
Stuttgart	300 km
Würzburg	170 km

Schätzungen der Restwegkosten für das Ziel *München*.

Finden Sie mit der **Gradienten-Suche** jeweils einen Weg von Würzburg nach München. Vergleichen Sie das Ergebnis mit der Best-First-Suche.

3.2. Search: Lokale Suche - Simulated Annealing

TL;DR

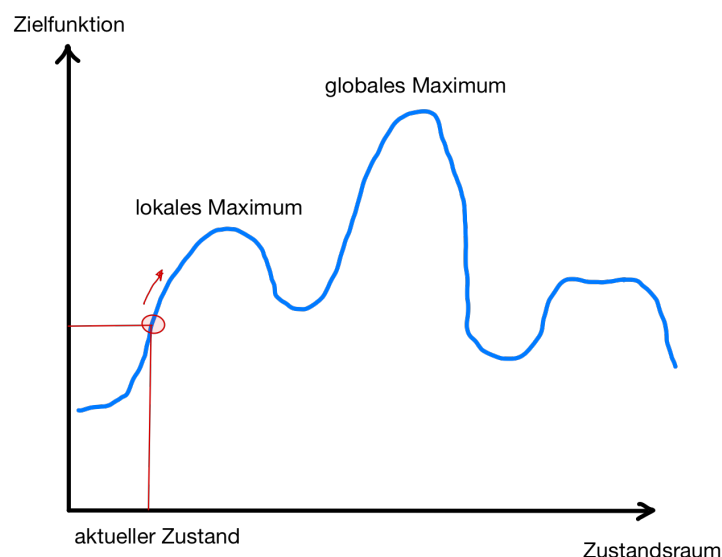
Lokale Suchverfahren: Nur das Ergebnis zählt! Nicht der Weg ist das Ziel, sondern nur das Erreichen des Ziels.

Das Problem bei der Gradientensuche ist, dass man eine Kostenfunktion benötigt und diese auch **lokale Minima** enthalten kann. Mit der reinen Gradientensuche würde man bei Erreichen lokaler Minima die Suche abbrechen (müssen), da es keine weitere Verbesserung unter den Nachfolgern mehr geben kann. In Anlehnung an das Abkühlen von Metall kann hier eine Variante der lokalen Suche helfen: **Simulated Annealing**. Man führt einen "Temperatur"-Parameter ein, der im Laufe der Suche immer kleiner wird und schließlich gegen Null geht. In Abhängigkeit von dieser "Temperatur" wird mit einer bestimmten Wahrscheinlichkeit eine Verschlechterung akzeptiert: Bei einer hohen Temperatur ist diese Wahrscheinlichkeit höher, bei einer niedrigen Temperatur niedriger, so dass das Verfahren in ein normales Hill-Climbing übergeht. Damit kann man ein Festfressen in lokalen Minima vermeiden bzw. überwinden.

Videos

- [VL Simulated Annealing](#)

3.2.1. Motivation



Problem: lokale Maxima und Plateaus

- Lokale Maxima/Minima: Algorithmus findet nur eine suboptimale Lösung
- Plateaus: Hier muss der Algorithmus mit zufälligen Zügen explorieren

Mögliche Lösungen:

- Neustart des Algorithmus, wenn kein Fortschritt erzielt wird
- Rauschen “injizieren”

3.2.2. Gedankenexperiment: Ausweg aus lokalen Minima

- “Drehen der Landschaft”: Minimieren statt Maximieren
- Ball wird in Zustandsraum-Landschaft gesetzt.
- Folge:
 - rollt steilsten Abstieg hinunter
 - rollt evtl. in Tal auf halber Höhe (lokales Minimum) => bleibt dort gefangen

=> “**Schütteln** der Landschaft” – Ball springt aus dem Tal und rollt in anderes Tal

Nicht zu stark schütteln – sonst wird u.U. globales Minimum verlassen!

3.2.3. Analogie Härten von Metall

- Metall erhitzen bis Atome frei beweglich
- Langsam abkühlen

=> stabiles Atomgitter mit minimalem Energiezustand

3.2.4. Übertragen der Idee

- Starkes “Schütteln” (hohe “Temperatur”) am Anfang
- Schrittweises “Abkühlen” => “Schütteln” im Laufe der Zeit verringern

=> **Simulated Annealing**

3.2.5. Pseudocode Simulated Annealing (Minimierungsproblem)

```
1 def simulated_annealing(problem):
2     current = problem.startNode
3     t = 0; temp = schedule(t)
4
5     while temp>0:
6         temp = schedule(++t)
7         neighbors = current.expandSuccessors()
8         if not neighbors: return current
```

```

9         working = random.choice(neighbors)
10        dE = problem.value(current) - problem.value(working)
11        if dE > 0 or probability(math.exp(dE/temp)):
12            current = working
13
14    return current

```

Wenn dE positiv ist, dann ist der Nachfolger “besser” (hier: kleiner bewertet) als der aktuelle Knoten und wird immer als nächster Knoten übernommen.

Wenn dE negativ ist, dann ist der betrachtete Nachfolger “schlechter” (hier: größer bewertet) als der aktuelle Knoten. Dann wird er mit einer Wahrscheinlichkeit $\text{math.exp}(dE/temp)$ als nächster Knoten übernommen. Diese Wahrscheinlichkeit ist bei hohen Temperaturen $temp$ eher hoch, und sinkt, je niedriger die Temperatur $temp$ wird.

Die Temperatur $temp$ bewegt sich dabei von hohen positiven Werten auf den Wert Null (wird also nicht negativ).

3.2.6. Detail: Akzeptieren von Verschlechterungen

Quelle: "Exp e.svg" by Marcel Marnitz, reworked by Georg-Johann on Wikimedia Commons (Public Domain)

- Wahrscheinlichkeit zum Akzeptieren einer Verschlechterung: $\text{math.exp}(dE/temp)$
- Wenn dE negativ ist, wird $\text{math.exp}(dE/temp)$ ausgewertet
- Wenn dE negativ, dann gilt (Umformung):

$$\exp(dE/temp) = \exp\left(-\frac{|dE|}{temp}\right) = \frac{1}{\exp\left(\frac{|dE|}{temp}\right)}$$

- Betrachtung von $\exp(a)$ bzw. e^a :
 - $a < 0$: geht gegen 0
 - $a = 0$: 1
 - $a > 0$: steil (exponentiell) gegen Unendlich ...
- Damit ergibt sich für dE (nur negativer Fall!) und $temp$:
 - Temperatur $temp$ hoch: $a = \frac{|dE|}{temp}$ ist positiv und klein (nahe Null), d.h. $\exp(a)$ nahe 1 (oder größer), d.h. die Wahrscheinlichkeit $1/\exp(a)$ ist nahe 1 (oder kleiner)
 - Temperatur $temp$ wird kleiner und geht gegen Null: $a = \frac{|dE|}{temp}$ ist positiv und wird größer, d.h. $\exp(a)$ geht schnell gegen Unendlich, d.h. die Wahrscheinlichkeit $1/\exp(a)$ geht gegen 0

3.2.7. Abkühlungsplan problemabhängig wählen

- Initiale Temperatur: So hoch, daß praktisch jede Änderung akzeptiert wird

- Abkühlen: $T_k = \alpha T_{k-1}$ mit $0.8 \leq \alpha \leq 0.99$
 - Ausreichend langsam abkühlen!
 - Typisch: jede Stufe so lange halten, daß etwa 10 Änderungen akzeptiert wurden
- Stop: Keine Verbesserungen in 3 aufeinander folgenden Temperaturen

Der Abkühlungsplan muss problemabhängig gewählt werden. Das Beispiel zeigt typische Elementes eines solchen Abkühlungsplans.

3.2.8. Eigenschaften Simulated Annealing

- Vollständigkeit: ja (mit gewisser Wahrscheinlichkeit)
- Optimalität: ja (mit gewisser Wahrscheinlichkeit)

Voraussetzung: geeigneter Abkühlungsplan

3.2.9. Anwendungen von Simulated Annealing

- Flugplan-Scheduling
- Layout-Probleme (Chipentwurf, Leiterplatten)
- Produktionsplanung

3.2.10. Wrap-Up

Lokale Suchverfahren: Nur das Ergebnis zählt!

- Gradientenverfahren
 - Analogie Bergsteigen: Gehe in Richtung des stärksten Anstiegs der Kostenfunktion => **Hill-Climbing**
 - Achtung: Probleme mit lokalen Minima => **Simulated Annealing**

Zum Nachlesen

- Russell und Norvig (2021): Simulated Annealing: Abschnitt 4.1.2

Lernziele

- k3: Ich kann die Funktionsweise von Simulated Annealing erklären und den Algorithmus auf ein konkretes Problem anwenden

Quizzes

- [Selbsttest Simulated Annealing \(ILIAS\)](#)

Challenges**Team-Planung**

Sie haben 12 Personen mit Rollen/Skills und paarweisen Synergiewerten (positiv = gut, negativ = Konflikt). Es gibt die Rollen/Skills "Frontend" (FE), "Backend" (BE), "Fullstack" (FS) sowie "User Experience" (UX). Für die Synergie-Werte überlegen Sie sich einige passende Beispiele, etwa "Person A und B kommen gut miteinander aus" o.ä.

Bilden Sie 3 Teams zu je 4 Personen, so dass

- die Summe der Synergien innerhalb der Teams maximiert wird,
- pro Team mindestens je 1x FE, 1x BE, 1x FS vorhanden ist (harte oder weiche Nebenbedingung),
- optional: UX in einem Team bringt Bonus.

Es gibt exakt 12 Personen, darunter jeweils 3 FE, 3 BE, 3 FS, 3 UX.

Modellieren Sie dieses Problem geeignet, damit Sie es mit Simulated Annealing lösen können.

4. Genetische Algorithmen

Lokale Suche mit Methoden, die der biologischen Evolution abgeschaut bzw. nachempfunden sind.

5. Spiele

Man kann Spiele auch als Suchproblem betrachten und als Ziel die Suche nach dem optimalen Zug definieren.

6. Entscheidungsbäume (Decision Tree Learner - DTL)

Beim überwachten Lernen soll eine Hypothese aufgebaut werden, die der echten (zu lernenden) Funktion möglichst nahe kommt. Eine Hypothese kann im einfachsten Fall als Entscheidungsbaum dargestellt werden. Die Merkmale bilden dabei die Knoten im Baum, und je Ausprägung gibt es eine Kante zu einem Nachfolgerknoten. Ein Merkmal bildet die Wurzel des Baums, an den Blättern sind die Klassen zugeordnet.

Einen Entscheidungsbaum kann man zur Klassifikation eines Objekts schrittweise durchlaufen: Für jeden Knoten fragt man die Ausprägung des Merkmals im Objekt ab und wählt den passenden Ausgang aus dem Knoten. Wenn man am Blatt angekommen ist, hat man die Antwort des Baumes auf das Objekt, d.h. üblicherweise die Klasse.

6.1. DTL: Machine Learning 101

TL;DR

Lernen wird in der KI oft als Verhaltensänderung (eines Systems) aufgefasst. Dabei soll eine Gütefunktion optimiert werden.

Je nach verfügbarem Feedback eines "Lehrers" werden typischerweise drei Arten von Lernen unterschieden: Überwachtes Lernen, Unüberwachtes Lernen, Reinforcement Lernen. Dabei stellt der Lehrer beim überwachten Lernen Trainingsbeispiele plus eine Vorgabe (Klasse, Funktionswert) zur Verfügung, während beim unüberwachten Lernen nur die Trainingsbeispiele bereitgestellt werden und der Algorithmus selbst Zusammenhänge in den Daten erkennen soll. Beim Reinforcement Learning erfolgt das Feedback am Ende einer Kette von Aktionen, d.h. der Algorithmus muss diese Bewertung auf die einzelnen Aktionen zurückrechnen.

Beim überwachten Lernen soll eine Hypothese aufgebaut werden, die der echten (zu lernenden) Funktion möglichst nahe kommt. Eine konsistente Hypothese erklärt die Trainingsdaten, eine generalisierende Hypothese kann auch unbekannte Daten (die aus der selben Quelle stammen, also zum selben Problem gehören) korrekt bewerten. Es wird unterschieden zwischen Klassifikation (einige wenige diskrete Label/Klassen, die den Trainingsbeispielen zugeordnet sind) und Regression (Lernen eines Funktionsverlaufs).

Merkmalsvektoren gruppieren Eigenschaften des Problems bzw. der Objekte, d.h. jedes Objekt kann über einen Merkmalsvektor beschrieben werden. Trainingsdaten sind ausgewählte Beispielobjekte (durch Merkmalsvektoren beschrieben) plus die Vorgabe (Klasse oder Funktionswert) vom Lehrer.

Videos

- [VL Machine Learning 101](#)

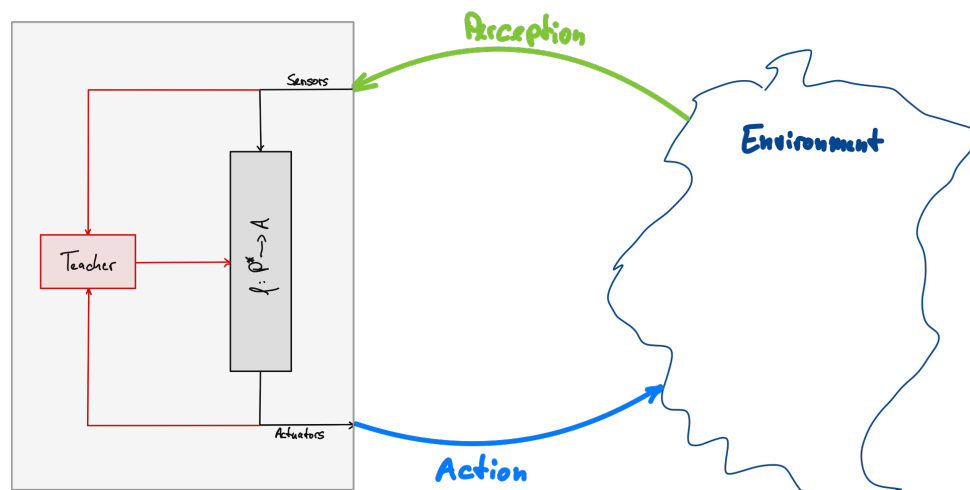
6.1.1. Was ist Lernen?

Verhaltensänderung eines Agenten in Richtung der Optimierung eines Gütefunktional (Bewertungsfunktion) durch Erfahrung.

6.1.2. Warum Lernen?

- Nicht alle Situationen vorhersehbar
- Nicht alle Details modellierbar
- Lösung oder Lösungsweg unbekannt, nicht explizit programmierbar
- Data Mining: Entdeckung neuen Wissens durch Analyse der Daten
- Selbstanpassende Programme

=> Lernen wichtige Eigenschaft lebender Wesen :-)

6.1.3. Learning Agent**6.1.4. Feedback während des Lernens**

- **Überwachtes Lernen**
 - Lernen durch Beobachtung
 - Vorgabe von Beispielen: Ein- und Ausgabewerte

=> Regression, Klassifikation

- **Unüberwachtes Lernen**

- Erkennen von Mustern in den Inputdaten, Clustering
- Kein Feedback (!)

- **Reinforcement Lernen**

- Bewertung der Aktionen des Agenten am Ende einer Aktionsfolge

Beispiel Kleinkind: Lernen von Klassen/Konzepten durch Beispiele

- Zuerst ist alles “Katze” (Übergeneralisierung)
- Differenzierung durch Feedback der Umwelt; Erkennung unterschiedlicher Ausprägungen

6.1.5. Beispiel: Kreditrisiko

- Bankkunde beantragt Kredit
- Soll er aus Sicht der Bank den Kredit bekommen?
- Bankangestellter betrachtet (relevante) Merkmale des Kunden:
 - Alter, Einkommen, sozialer Status
 - Kundenhistorie bei der Bank
 - Höhe des Kredits
- Bewertung des Kreditrisikos:
 - **Klassifikation:** Guter oder schlechter Kunde (Binäre Entscheidung: 2 Klassen)
 - **Regression:** Vorhersage Gewinn/Verlust für die Bank (Höhe des Gewinns/Verlusts interessant)

6.1.6. Beispiel: Autoreparatur

- **Gegeben:** Eigenschaften eines Autos
=> Eigenschaften: Ausprägungen der Merkmale
- **Gesucht:** Diagnose und Reparaturanleitung
=> Hypothese über den Merkmalen (Funktion h)

6.1.7. Lernen durch Beobachten: Lernen einer Funktion f

Funktionsapproximation: Lernen einer Funktion f anhand von Beispielen

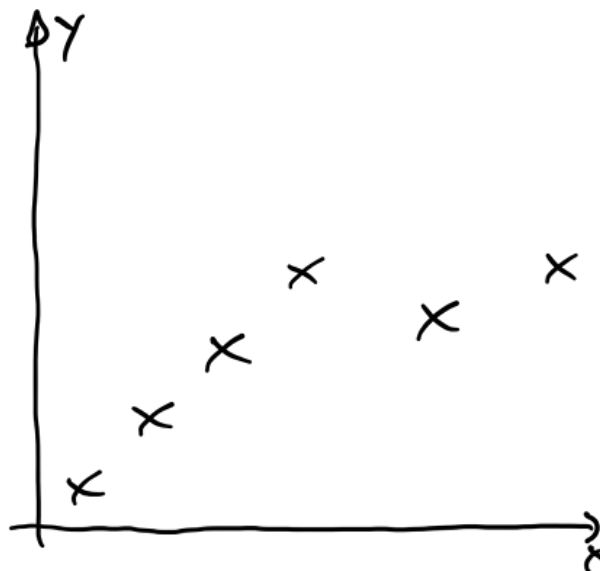
- Ein Beispiel ist ein Tupel $(x, f(x))$, etwa

$$(x, f(x)) = \left(\begin{pmatrix} 0 & 0 & X \\ . & X & . \\ X & . & . \end{pmatrix}, +1 \right)$$

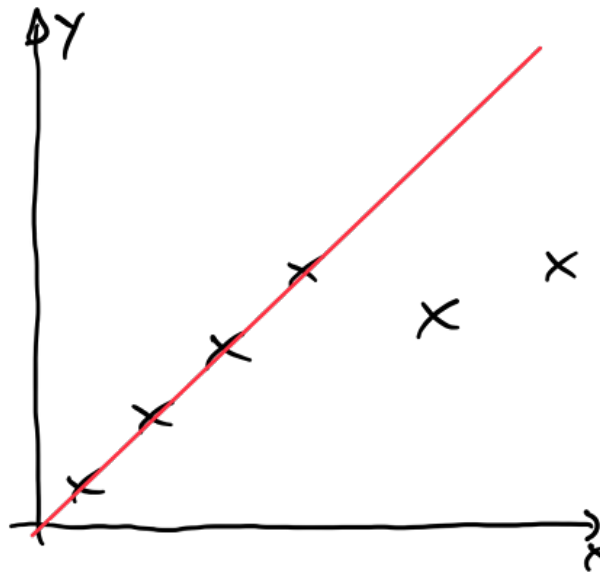
- Aufgabe: Baue Hypothese h auf, so dass $h \approx f$.
 - Benutze dazu Menge von Beispielen $\Rightarrow$ **Trainingsdaten**.
- Ziele:
 1. **Konsistente Hypothese**: Übereinstimmung bei Trainingsdaten
 2. **Generalisierende Hypothese**: Korrekte Vorhersage bei unbekannten Daten

Anmerkung: Stark vereinfachtes Modell realen Lernens!

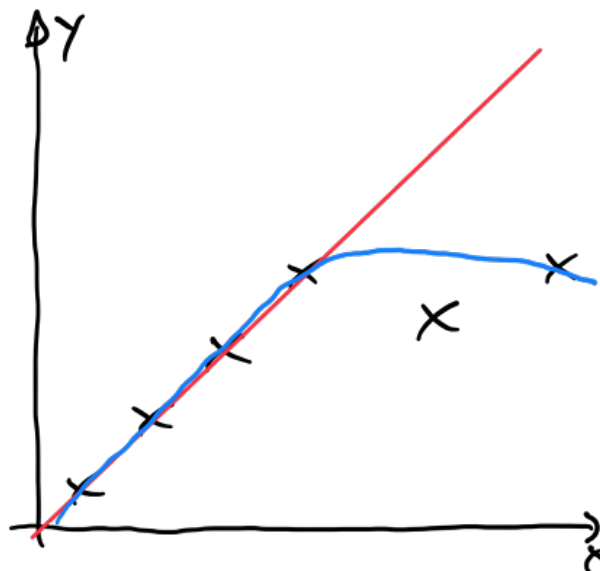
6.1.8. Konstruieren einer konsistenten Hypothese



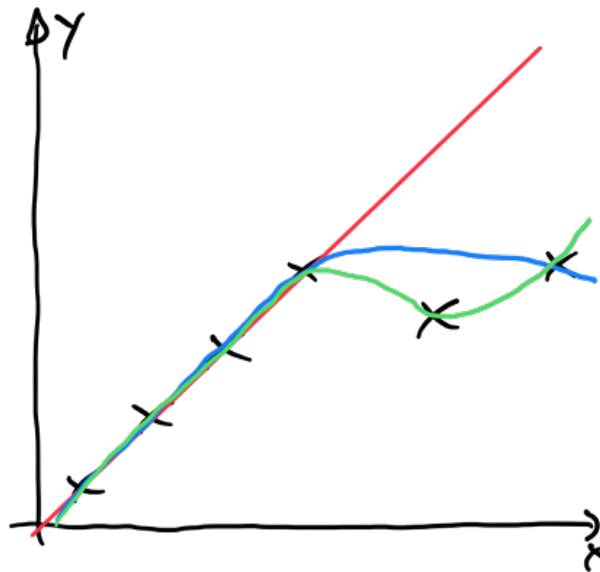
Welcher Zusammenhang ist hier dargestellt? Offenbar eine Art Funktionsverlauf ... Wir haben für einige x -Werte die zugehörigen y -Werte vorgegeben.

6.1.9. Konstruieren einer konsistenten Hypothese (cnt.)

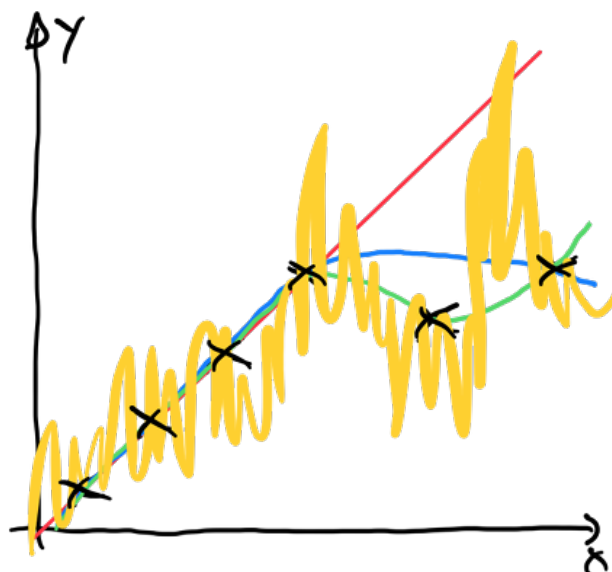
Die einfachste Approximation wäre eine lineare Funktion. Allerdings werden hierbei einige Werte mehr oder weniger stark nicht korrekt wiedergegeben, d.h. man hat einen relativ hohen (Trainings-) Fehler.

6.1.10. Konstruieren einer konsistenten Hypothese (cnt.)

Die Hyperbel erklärt die Trainingsdaten bis auf den einen Punkt sehr gut. Die Frage ist, ob dieser eine Punkt zum zu lernenden Zusammenhang gehört oder ein Ausreißer ist, den man gefahrlos ignorieren kann?

6.1.11. Konstruieren einer konsistenten Hypothese (cnt.)

Die grüne Hypothese ist von allen bisher gezeigten die komplexeste, erklärt aber alle Datenpunkte. D.h. hier wäre der Trainingsfehler Null. Zwischen den Trainingsdaten zeigt das Modell eine "glatte" Approximation, d.h. es wird auch neue Daten, die es beim Training nicht gesehen hat, relativ gut erklären. (Dabei liegt freilich die Annahme zugrunde, dass alle relevanten Daten in der Trainingsmenge vorhanden sind, d.h. dass es insbesondere zwischen den Datenpunkten keine Ausreißer o.ä. gibt.)

6.1.12. Konstruieren einer konsistenten Hypothese (cnt.)

Diese Hypothese erklärt ebenfalls sämtliche Trainingsdaten. Allerdings schwingt die Funktion zwischen den Daten stark hin und her. Vermutlich entspricht dies nicht dem zu lernenden Funktionsverlauf. Der Trainingsfehler wäre wie bei der deutlich einfacheren Hypothese aus dem letzten Schritt Null. Der Generalisierungsfehler (sprich die Abweichung, wenn man das Modell nach Daten zwischen den Trainingspunkten fragt) dürfte erheblich höher liegen.

D.h. hier hat das Modell einfach die Trainingsdaten auswendig gelernt, aber nicht den Zusammenhang zwischen den Daten! Dies ist in der Regel unerwünscht!

6.1.13. Occam's Razor

Bevorzuge die einfachste konsistente Hypothese!

1. Wenn es mehrere mögliche Erklärungen für einen Sachverhalt gibt, ist die einfachste Erklärung allen anderen vorzuziehen.
2. Eine Erklärung ist "einfach", wenn sie möglichst wenige Variablen und Annahmen enthält und wenn diese in klaren logischen Beziehungen zueinander stehen, aus denen der zu erklärende Sachverhalt logisch folgt.

6.1.14. Trainingsdaten und Merkmalsvektoren

Lehrer gibt Beispiele vor: Eingabe x und passende Ausgabe $f(x)$

- Ausgabe: typischerweise Skalar (Funktionswert oder Klasse)
=> Beispiel: Bewertung eines Spielstandes bei TicTacToe
- Eingabe: (Beschreibung des) Objekt(s) oder Situation, die zur Ausgabe gehört
=> Beispiel: Spielstand bei TicTacToe

Merkmalsvektoren:

- Zusammenfassen der relevanten Merkmale zu Vektoren

6.1.15. Beispiel: Schwimmen im See

Beschreibung der Faktoren, wann ich im See schwimmen möchte:

1. Scheint die Sonne?
2. Wie warm ist das Wasser?
3. Wie warm ist die Luft?

- Trainingsbeispiel:

- Eingabe: Merkmalsvektor (**sonnig**, **warm**, **warm**)
- Ausgabe: Klasse **ja**

Dabei wird davon ausgegangen, dass jeder Faktor (jedes Merkmal) an einer bestimmten Stelle im Merkmalsvektor aufgeführt ist. Beispielsweise gehört das **sonnig** zur Frage “Scheint die Sonne”, **warm** jeweils zur Wasser- und zur Lufttemperatur.

Damit hat man in einem Vektor eine Situation komplett beschrieben, d.h. einen Zustand der Welt mit den relevanten Dingen beschrieben. Diesem Zustand kann man beispielsweise ein Label (Klasse) verpassen, hier in diesem Fall “ja, in dieser Welt möchte ich schwimmen”.

Die Trainingsmenge baut sich dann beim überwachten Lernen aus vielen solcher Paare (Merkmalsvektor, Klasse) auf, und die Algorithmen sollen diese Zuordnung lernen, d.h. ein Modell für diese Daten erzeugen, welches die Daten gut erklärt und darüber hinaus für neue Daten aus der selben Datenquelle gute Vorhersagen macht.

6.1.16. Trainingsdaten – Merkmalsvektoren

Generell: Merkmalsvektor für Objekt v :

$$\mathbf{x}(v) = (x_1, x_2, \dots, x_n)$$

- n Merkmale (Attribute)
- Attribut x_t hat m_t mögliche Ausprägungen
- Ausprägung von v bzgl. x_t : $x_t(v) = i$ (mit $i = 1 \dots m_t$)

Anmerkung: Stellen Sie sich den Merkmalsvektor $\mathbf{x}$ vielleicht wie einen Konstruktor einer Klasse $\mathbf{x}$ vor: Die einzelnen Attribute x_t sind die Parameter, aus denen der Merkmalsvektor aufgebaut ist/wird. Jedes der Attribute hat einen Typ und damit eine bestimmte Anzahl erlaubter Werte (“Ausprägungen”) ...

Trainingsbeispiel:

- Tupel aus Merkmalsvektor und zugehöriger Klasse: $(\mathbf{x}(v), k)$

6.1.17. Wrap-Up

- Lernen ist Verhaltensänderung, Ziel: Optimierung einer Gütefunktion
 - Aufbau einer Hypothese, die beobachtete Daten erklären soll
 - Arten: Überwachtes Lernen, Unüberwachtes Lernen, Reinforcement Lernen
- Merkmalsvektoren gruppieren Eigenschaften des Problems bzw. der Objekte
- Trainingsdaten: Beispielobjekte (durch Merkmalsvektoren beschrieben) plus Vorgabe vom Lehrer

Zum Nachlesen

- Russell und Norvig (2021): Lernen: Abschnitte 19.1 und 19.2

Lernziele

- k1: Ich kann den Begriff “(maschinelles) Lernen” definieren und verschiedene Arten unterscheiden
- k2: Ich kann “Überwachtes Lernen” erklären: Lernen durch Beobachten (mit Lehrer)
- k2: Ich kann wichtige Elemente des (maschinellen) Lernens erklären: Merkmalsvektoren, Eigenschaften, Ausprägung, Objekte, Trainingsmenge

Quizzes

- [Selbsttest Intro ML \(ILIAS\)](#)

Challenges**Modellierung**

Sie stehen vor der Entscheidung, ob Sie sich zur Vorbereitung auf die Flipped-Classroom-Sitzung noch das Skript anschauen. Welche Attribute benötigen Sie, um die Situation zu beschreiben?

Metriken für Klassifikatoren

Es ist wieder Wahlkampf: Zwei Kandidaten O und M bewerben sich um die Kanzlerschaft. Die folgende Tabelle zeigt die Präferenzen von sieben Wählern.

Nr.	Alter	Einkommen	Bildung	Kandidat	Vorhersage
1	≥ 35	hoch	Abitur	O	O
2	< 35	niedrig	Master	O	O
3	≥ 35	hoch	Bachelor	M	M
4	≥ 35	niedrig	Abitur	M	M
5	≥ 35	hoch	Master	O	O
6	< 35	hoch	Bachelor	O	M
7	< 35	niedrig	Abitur	M	O

Auf diesem Datensatz wurde ein Klassifikator trainiert, die Trainingsergebnisse sind in der Tabelle unter “Vorhersage” angegeben.

Bewerten Sie den Klassifikator.

7. Constraintsolving

Was haben [Typ-Inferenz](#), [Sudoku](#), das [8-Queens-Problem](#), das [Einfärben von Landkarten](#) und das [Erstellen von Stundenplänen](#) gemeinsam?

Es handelt sich um eine bestimmte Art von Suchproblemen, wobei den Parametern (Variablen) Werte so zugewiesen werden müssen, dass Einschränkungen bzw. Relationen zwischen den Variablen gelten.

8. NN: Einführung in Neuronale Netze

Das Perzeptron kann als die Nachahmung einer biologischen Nervenzelle betrachtet werden. Durch das Zusammenschließen dieser “künstlichen” Nervenzellen entstehen künstliche **Neuronale Netze** (NN), die ähnlich wie das Gehirn **lernen** sollen, komplexe Aufgaben zu bewerkstelligen.

8.1. NN01: Das Perzeptron

Videos

- [NN1.1 - Einführung](#)
- [NN1.2 - Fallstudie und Formalisierung](#)
- [NN1.3 - Das Perzeptron Modell](#)
- [NN1.4 - Perzeptron Beispiel](#)
- [NN1.5 - Der Perzeptron Lernalgorithmus](#)

Weitere Unterlagen

- [NN01-Das_Perzeptron.pdf](#)
- [Perzeptron_Lernalgorithmus.ipynb](#)

8.1.1. Kurze Übersicht

8.1.1.1. Definition “Maschinelles Lernen”

Fähigkeit zu lernen, ohne explizit programmiert zu werden. (Arthur Samuel, 1959)

8.1.1.2. Arten des Lernens

- Überwachtes Lernen (e.g. Klassifizierung, Regression)
- Unüberwachtes Lernen (e.g. Clustering, Dimensionsreduktion)
- Bestärkendes Lernen (e.g. Schach spielen)

Vergleiche auch DTL: Machine Learning 101.

8.1.1.3. Formalisierung

- Zielfunktion f
- Merkmalraum (input space)
- Ausgaberaum (output space)
- Datensatz $\mathcal{D}$
- Hypothesenmenge $\mathcal{H}$
- Lernalgorithmus $\mathcal{A}$

8.1.1.4. Das Perzeptron

Das Perzeptron ist ein einfaches Modell für die **binäre Klassifizierung**.

Hypothesenfunktion

- Die **Hypothesenfunktion** ist eine gewichtete Summe der Eingabemerkmale x_i plus eine Konstante w_0 , auf die als Aktivierung die Signumfunktion angewendet wird:

$$h(\mathbf{x}) = \text{sgn}(w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n)$$

$$= \text{sgn}(\mathbf{w}^T \mathbf{x})$$

$$= \begin{cases} 1, & \text{wenn } \mathbf{w}^T \mathbf{x} \geq 0, \\ -1, & \text{wenn } \mathbf{w}^T \mathbf{x} < 0 \end{cases}$$

- Die **Ausgabe** $\hat{y}$ des Modells ist somit eine Zahl aus der Menge $\{1, -1\}$.
- $w_0, w_1, w_2, \dots, w_n$ sind die Parameter des Modells, die es zu optimieren gilt.

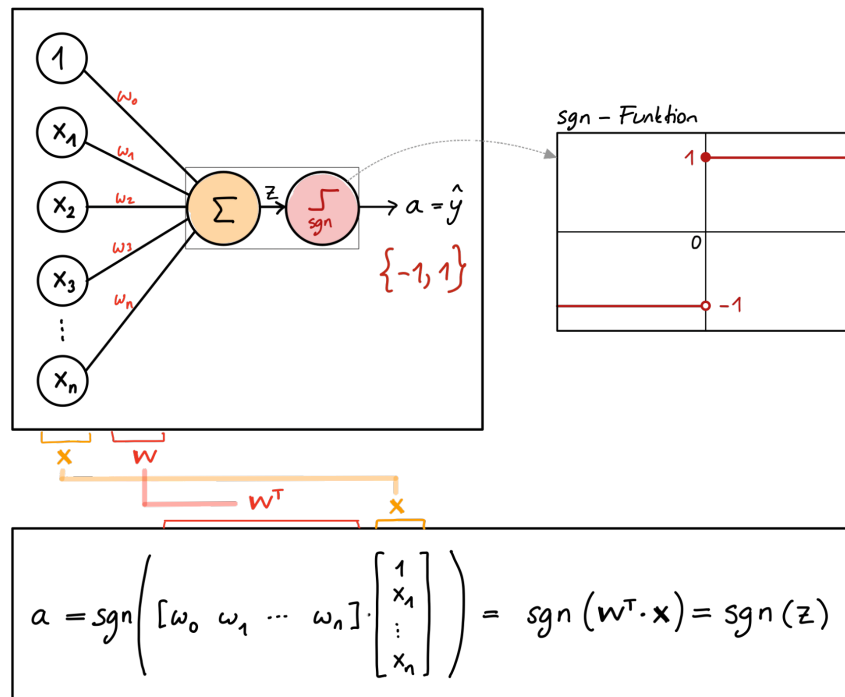


Abbildung 8.1.: Perzeptron Aufbau und Berechnung

Lineare Entscheidungsgrenze

- Das Perzeptron erzeugt eine **lineare Entscheidungsgrenze** im Merkmalraum, die durch die folgende Gleichung definiert ist:

$$\mathbf{w}^T \mathbf{x} = w_0 + w_1 x_1 + w_2 x_2 + \dots + w_n x_n = 0$$

- Für einen gegebenen Datensatz müssen nun jene Parameter identifiziert werden, die eine Entscheidungsgrenze definieren, die alle Datenpunkte korrekt klassifiziert. Dies ist natürlich nur dann möglich, sofern die Daten **linear separierbar** sind.
- Im zweidimensionalen Merkmalraum $\mathbb{R}^2$ entspricht die Entscheidungsgrenze einer **Geraden**. Die folgende Abbildung zeigt beispielsweise die Entscheidungsgrenze für die Parameter $w_0 = 0, w_1 = 2$ und $w_3 = -1$, entsprechend dem Modell $h(\mathbf{x}) = \text{sgn}(2x_1 - x_2)$.

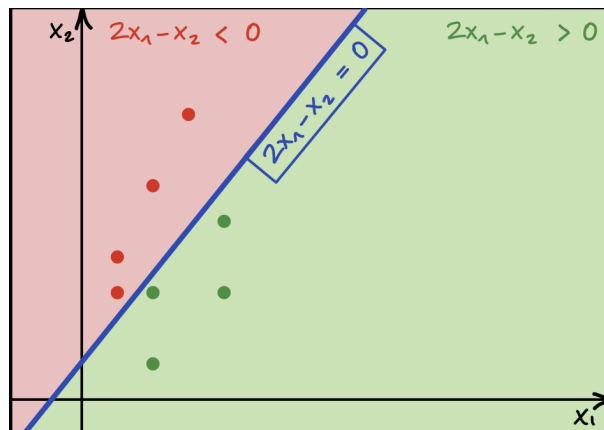


Abbildung 8.2.: Perzeptron Entscheidungsgrenze

8.1.1.5. Der Perzeptron Lernalgorithmus

Der Perzeptron Lernalgorithmus (PLA) startet mit zufälligen Gewichtsparametern und aktualisiert methodisch die Gewichte, bis die Daten alle richtig klassifiziert sind (oder eine festgelegte obere Schranke von Aktualisierungsschritten erreicht wird).

Ablauf des PLA:

1. Starte mit zufälligen Gewichtsparametern $\mathbf{w}$.
2. Wiederhole die folgenden Schritte, solange es falsch klassifizierte Punkte gibt:
 - Wähle einen beliebigen, falsch klassifizierten Punkt $\mathbf{x}$ aus. Sei y das zugehörige Label (Zielwert).
 - **Aktualisiere den Gewichtsvektor** wie folgt:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \cdot y \cdot \mathbf{x}$$

Hier bezeichnet α die **Lernrate** (auch Schrittweite genannt). Sie steuert, wie stark die Parameter in jedem Schritt in Richtung $y \cdot \mathbf{x}$ aktualisiert werden, und beeinflusst somit maßgeblich die Konvergenzgeschwindigkeit und Stabilität des Lernprozesses. Es existiert kein universell optimaler Wert für α ; dieser muss problemabhängig empirisch ermittelt werden. Typische Startwerte für Experimente liegen bei 0.1, 0.01 oder 0.001.

Die **Gewichtsaktualisierung** ist der Ort, wo das Lernen stattfindet.

8.1.1.6. Nicht-Lineare Entscheidungsgrenze

Um **nicht-lineare Entscheidungsgrenzen** zu ermöglichen, können die Originalmerkmale auf nicht-lineare Weise kombiniert (z. B. $x_1^2, x_2^2, x_1x_2, \dots$) und diese neuen Werte als zusätzliche Merkmale in den Vektor $\mathbf{x}$ aufgenommen werden. Die Hypothesenfunktion bleibt dabei weiterhin **linear in den Gewichten** w_i .

Im Fall von zwei Merkmalen kann beispielsweise die folgende Hypothesenfunktion verwendet werden:

$$h(\mathbf{x}) = \text{sgn}(w_0 + w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_2^2).$$

Durch die Einbeziehung quadratischer Terme lassen sich nichtlineare Entscheidungsgrenzen modellieren, darunter insbesondere kreisförmige Entscheidungsgrenzen der Form

$$-4 + x_1^2 + x_2^2 = 0.$$

Lernziele

- k2: Ich kann verschiedene Arten des maschinellen Lernens erklären
- k2: Ich kann die Formalisierung eines ML-Problems, insbesondere der Klassifizierung erklären und dabei insbesondere die Begriffe Datensatz, Merkmalsraum, Hypothesenfunktion und Zielfunktion an einem Beispiel erläutern
- k2: Ich kann das Perzeptron als linearen Klassifikator verstehen und erklären
- k2: Ich kann den Begriff der Entscheidungsgrenze erklären
- k3: Ich kann die Entscheidungsgrenze für ein konkretes Beispiel berechnen
- k3: Ich kann den Perzeptron Lernalgorithmus implementieren und anwenden

Quizzes

- [Selbsttest Intro ML \(ILIAS\)](#)

8.2. NN02: Lineare Regression und Gradientenabstieg

Videos

- [NN2.1 - Lineare Regression - Intro](#)
- [NN2.2 - Vergleich Perzeptron und Bsp](#)
- [NN2.3 - Kostenfunktion und Gradientenvektor](#)
- [NN2.4 - Berechnung Gradientenvektor - Beispiel](#)
- [NN2.5 - Berechnung Gradientenvektor - Allgemein](#)
- [NN2.6 - Skalierung der Merkmale](#)

Weitere Unterlagen

- [NN02-Lineare_Regression.pdf](#)
- [Lineare_Regression.ipynb](#)

8.2.1. Kurze Übersicht

8.2.1.1. Formalisierung

- Ausgabe y ist reelle Zahl aus einem stetigen Bereich (zum Beispiel Hauspreis)

- Die **Hypothesenfunktion** ist eine gewichtete Summe der Merkmale x_i plus eine Konstante w_0 :

$$h(\mathbf{x}) = \mathbf{w}^T \mathbf{x} = w_0 + w_1 x_1 + w_2 x_2 + \dots + w_n x_n$$

- Der **Verlust** (engl. loss) für einen Datenpunkt $\mathbf{x}$ ist das **Fehlerquadrat**:

$$\mathcal{L} = (\hat{y} - y)^2 = (h(\mathbf{x}) - y)^2$$

- Die Kosten (engl. cost) sind der durchschnittliche Verlust über alle Datenpunkte:

$$J = \frac{1}{2m} \sum_{i=1}^m (\hat{y} - y)^2 = \frac{1}{2m} \sum_{i=1}^m (h(\mathbf{x}) - y)^2$$

8.2.1.2. Der Gradient

- Der **Gradientenvektor** $\nabla J(\mathbf{w})$ setzt sich zusammen aus den partiellen Ableitungen der Kostenfunktion J nach den Gewichten w_i und zeigt in jedem Punkt $\mathbf{w}$ in die **Richtung des steilsten Aufstiegs**:

$$\nabla J = [\partial J / \partial w_0 \quad \partial J / \partial w_1 \quad \dots \quad \partial J / \partial w_n]^T$$

- **Schlussfolgerung:** In die entgegengesetzte Richtung, i.e. in Richtung $-\nabla J(\mathbf{w})$ geht es am *steilsten bergab*!
- **IDEE:** Bewege $\mathbf{w}$ in Richtung $-\nabla J(\mathbf{w})$, um die Kosten J möglichst schnell zu senken.

8.2.1.3. Der Gradientenabstieg (engl. Gradient Descent)

1. Starte mit zufälligen Gewichten $\mathbf{w}$
2. Berechne den Gradientenvektor im aktuellen Punkt $\mathbf{w}$
3. **Gewichtsaktualisierung:** Gehe einen *kleinen* Schritt in Richtung $-\nabla J(\mathbf{w})$

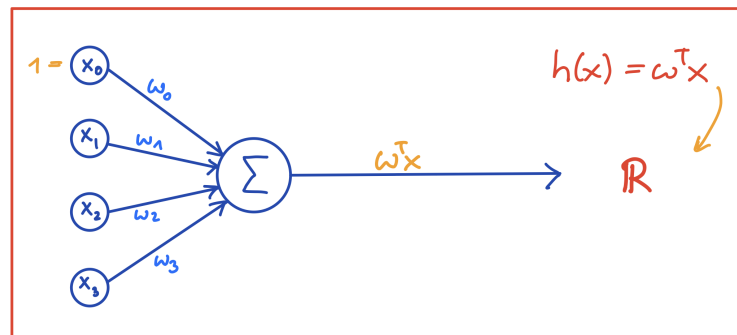
$$\mathbf{w}_{neu} := \mathbf{w}_{alt} - \alpha \cdot \nabla J(\mathbf{w}_{alt})$$

(α : Lernrate/Schrittweite).

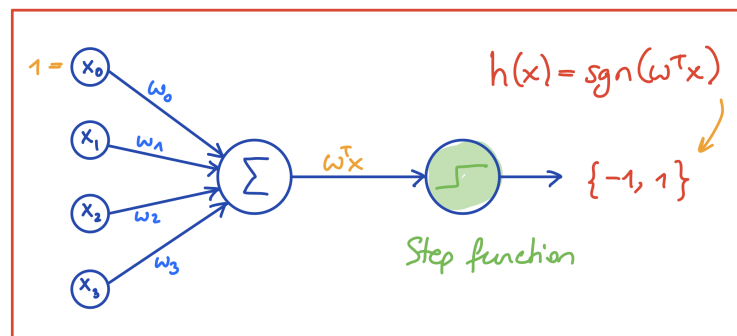
4. Wiederhole Schritte 2-3, bis das globale Minimum von J erreicht ist.

8.2.1.4. Graphische Übersicht

- **Lineare Regression**



• Perzeptron



Lernziele

- k2: Ich kann die lineare Regression aus Sicht neuronaler Netze erklären und dabei auf die graphische Darstellung und den Vergleich mit dem Perzeptron eingehen
- k2: Ich kann die Formalisierung von linearer Regression erklären
- k2: Ich kann den Begriff Verlust- und Kostenfunktion erklären
- k2: Ich kann den Gradientenvektor an einem Beispiel erklären
- k2: Ich kann den Begriff Lernrate erklären
- k3: Ich kann den Gradientenabstieg an einem Beispiel anwenden

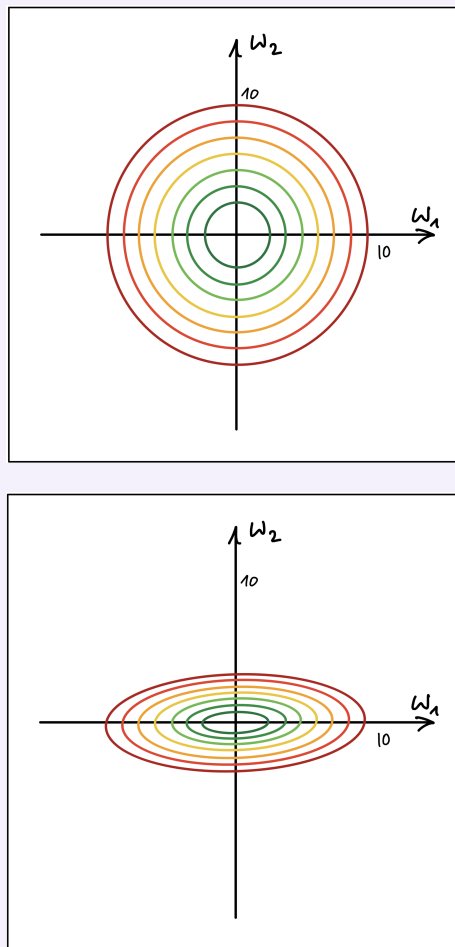
Quizzes

- [Selbsttest Lineare Regression \(ILIAS\)](#)

Challenges

Skalierung der Merkmale

Abbildung 1 und Abbildung 2 zeigen die Höhenlinien (Contour Lines) von zwei Kostenfunktionen.



- Erklären Sie, welcher der beiden Fälle nachteilhaft für den Gradientenabstieg Algorithmus ist. Wo liegt der Nachteil? Wie kann die Merkmalskalierung dem genannten Nachteil entgegenwirken?
- Zeigen Sie unter Verwendung Ihrer eigenen, zufällig generierten Datenpunkte aus dem Bereich $[100, 300] \times [0, 2]$, wie sich Standardisierung, Min-Max Skalierung und Normalisierung auf die Daten auswirken. Vergleichen Sie dazu die jeweiligen Streudiagramme (scatterplots). Sie können hierzu das folgende [Jupyter Notebook](#) als Startpunkt benutzen.

8.3. NN03: Logistische Regression

Videos

- [NN3.1 - Logistische Regression - Intro](#)
- [NN3.2 - Logistische Regression - Hypothesenfunktion und Bsp](#)
- [NN3.3 - Logistische Regression - Verlust und Kosten](#)
- [NN3.4 - Logistische Regression - Gradientenabstieg](#)

Weitere Unterlagen

- [NN03-Logistische_Regression.pdf](#)
- [Logistische_Regression.ipynb](#)

8.3.1. Kurze Übersicht**8.3.1.1. Formalisierung**

- Ausgabe y ist reelle Zahl aus dem stetigen Bereich $(0, 1)$
- Die **Hypothesenfunktion** ist:

$$h(\mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x}) = \sigma(w_0 + w_1 x_1 + w_2 x_2 + \dots + w_n x_n) \quad (1)$$

- Der **Kreuzentropie Verlust** (engl. Cross-Entropy) für einen Datenpunkt $\mathbf{x}$:

$$\mathcal{L}(a, y) = -y \log(a) - (1 - y) \log(1 - a) \quad (2)$$

wobei hier $a := \hat{y}$ die Vorhersage ist.

- Die Kosten als durchschnittlicher Verlust über alle Datenpunkte $x^{(1)}, \dots, x^{(m)}$:

$$J = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(a^{(i)}, y^{(i)}) \quad (3)$$

8.3.1.2. Gradientenabstieg

- Der Gradient für einen Datenpunkt $\mathbf{x}$:

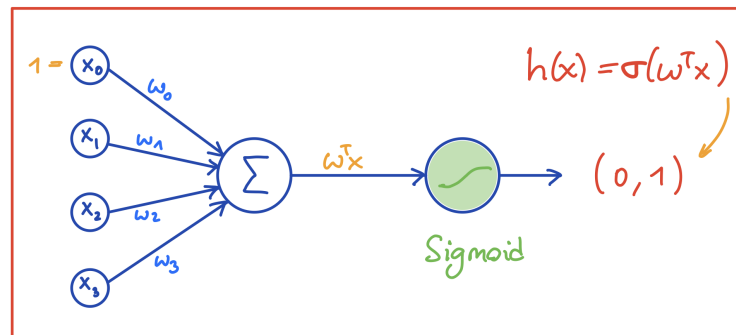
$$\frac{\partial \mathcal{L}}{\partial w} = (a - y)x \quad (4)$$

- Der Gradient für alle Datenpunkte X in Matrix-Notation:

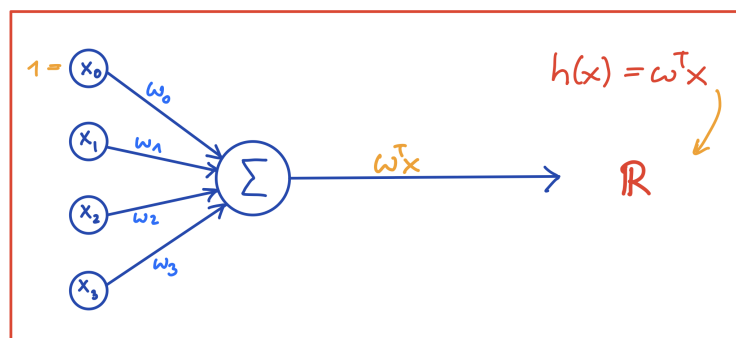
$$\nabla J = \frac{\partial J}{\partial w} = \frac{1}{m} X(A - Y)^T \quad (5)$$

8.3.1.3. Graphische Übersicht

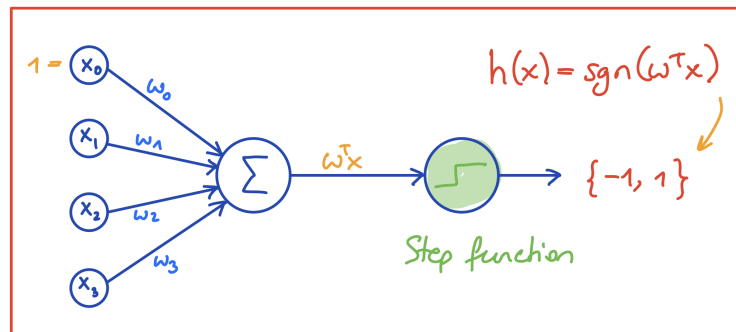
- **Logistische Regression**



• Lineare Regression



• Perzeptron



Lernziele

- k2: Ich kann die logistische Regression aus Sicht neuronaler Netze erklären und dabei auf die graphische Darstellung und den Vergleich mit Perzeptron und linearer Regression eingehen
- k2: Ich kann die Formalisierung der logistische Regression erklären
- k2: Ich kann die Sigmoid-Aktivierungsfunktion erklären und an einem Beispiel demonstrieren
- k2: Ich kann Verlust- und Kosten (Cross-Entropy Loss) erklären
- k3: Ich kann den Gradientenabstieg für logistische Regression an einem Beispiel anwenden

Quizzes

- [Selbsttest Logistische Regression \(ILIAS\)](#)

8.4. NN04: Overfitting und Regularisierung

Videos

- [NN4.1 - Nichtlineare Modelle](#)
- [NN4.2 - Overfitting und Regularisierung](#)

Weitere Unterlagen

- [NN04-Nichtlineare_Modelle_und_Overfitting.pdf](#)

8.4.1. Kurze Übersicht

8.4.1.1. Nichtlineare Modelle

- Einführung von neuen Merkmalen in Form von nichtlinearen Kombinationen der ursprünglichen Merkmale
- Erhöhung der Komplexität des Modells ermöglicht das Erfassen von nichtlinearen Beziehungen
- **Bemerkung:** Die Hypothesenfunktion bleibt linear in den Gewichten, es wird weiterhin logistische Regression in einem **erweiterten** Merkmalraum durchgeführt.

8.4.1.2. Überanpassung und Regularisierung

- Die **Überanpassung** (engl. Overfitting) ist eines der häufigsten und wichtigsten Probleme in ML und DL

“Was im Bereich des maschinellen Lernens Professionelle von Amateuren unterscheidet, ist ihre Fähigkeit mit Überanpassung umzugehen.”

— Quelle: (Abu-Mostafa u. a. 2012, 119)

- Anzeichen von Überanpassung sind geringe Trainingskosten und hohe **Testkosten** (Kosten auf nicht-gesehenen Daten).
- Regularisierung ist eine Maßnahme gegen Überanpassung. Man kann es sich als eine Reduktion in der Komplexität des Modells vorstellen.
- Der Regularisierungsparameter λ ist ein Hyperparameter. Je größer der λ -Wert, desto größer der Regularisierungseffekt.
- Die **Kostenfunktion** bei regularisierter logistischer Regression:

$$J = \frac{1}{m} \left[\sum_{i=1}^m \left(-y^{[i]} \log(a^{[i]}) - (1 - y^{[i]}) \log(1 - a^{[i]}) \right) + \frac{\lambda}{2} \sum_{j=1}^n (w_j^2) \right] \quad (1)$$

- Die **Gewichtsaktualisierung** mit Regularisierungsterm:

$$w_j := w_j - \frac{\alpha}{m} \left[\sum_{i=1}^m \left((a^{[i]} - y^{[i]}) x_j^{[i]} \right) + \lambda w_j \right] \quad (2)$$

Zum Nachlesen

- Abu-Mostafa u. a. (2012): Kapitel 4

Lernziele

- k2: Ich kann die Erhöhung der Modell-Komplexität durch Einführung von Merkmalen höherer Ordnung erklären
- k2: Ich kann die Begriffe Unter- und Überanpassung an einem Beispiel erklären
- k2: Ich kann den Begriff Regularisierung erklären und die Auswirkung auf die Gewichte und das Modell erläutern
- k3: Ich kann den Gradientenabstieg für regularisierte logistische Regression an einem Beispiel anwenden

Quizzes

- [Selbsttest Overfitting \(ILIAS\)](#)

8.5. NN05: Multilayer Perzeptron

Videos

- [NN5.1 - MLP Forward Propagation](#)

Weitere Unterlagen

- [NN05-MLP.pdf](#)

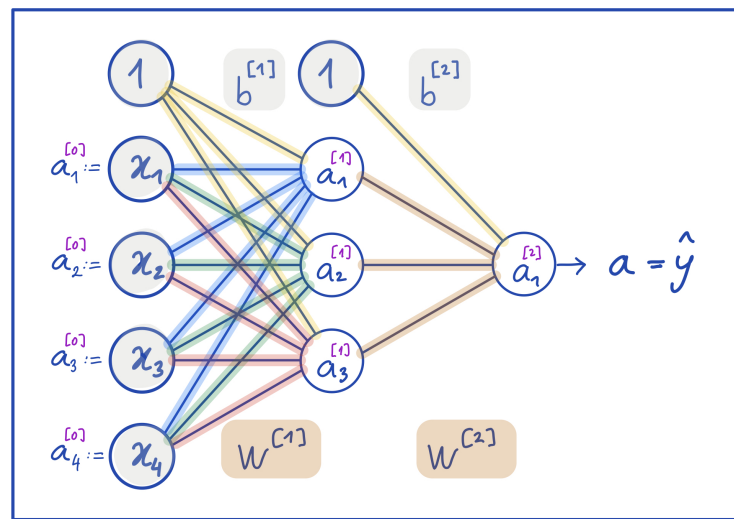
8.5.1. Kurze Übersicht

8.5.1.1. Multilayer Perzeptron (MLP)

- Das Perzeptron kann nur linear separable Daten korrekt klassifizieren.
- Durch das Zusammenschließen von mehreren Perzeptronen kann man ein mehrschichtiges Perzeptron (engl. Multilayer Perceptron) aufstellen, das komplexere Funktionen modellieren kann.
- Ein MLP wird oft auch als **Feed Forward Neural Network** oder als **Fully Connected Neural Network** bezeichnet.
- Die “inneren” Schichten eines solchen Netzwerkes sind sogenannte **versteckte Schichten** (engl. hidden layer). Das sind alle Schichten ausgenommen die Eingangs- und Ausgangsschicht.

8.5.1.2. Graphische Übersicht und Vorwärtslauf

- Ein Multi-Layer Perzeptron



Ein Vorwärtslauf (forward pass):

$$a^{[1]} = \text{ReLU} \left(W^{[1]} \cdot x + b^{[1]} \right) \quad (1)$$

$$\hat{y} := a^{[2]} = \sigma \left(W^{[2]} \cdot a^{[1]} + b^{[2]} \right) \quad (2)$$

Lernziele

- k2: Ich kann das Multi-Layer Perzeptron (MLP) erläutern und dabei auf die graphische Darstellung eingehen und den Vorwärtslauf erklären
- k2: Ich kann verschiedene Aktivierungsfunktionen (insbesondere ReLU) erklären
- k3: Ich kann den Vorwärtslauf (forward pass) für ein gegebenes MLP anwenden
- k3: Ich kann die Berechnung der einzelnen Aktivierungen durchführen

Quizzes

- [Selbsttest Multilayer Perzeptron \(ILIAS\)](#)

Challenges

Lineares MLP

Gegeben sei ein MLP mit linearen Aktivierungsfunktionen, d.h. für jedes Neuron berechnet sich der Output durch die gewichtete Summe der Inputs: $y = g(w^T x)$, wobei $g(z) = z$ gilt, also $y = w^T x$. Zeigen Sie, dass dieses Netz durch eine einzige Schicht mit linearen Neuronen ersetzt werden kann.

Betrachten Sie dazu ein zwei-schichtiges Netz (i.e. bestehend aus Eingabe-Schicht, Ausgabe-Schicht und einer versteckten Schicht) und schreiben Sie die Gleichung auf, die die Ausgabe als Funktion der Eingabe darstellt.

Als Beispiel sei das zwei-schichtige MLP mit den folgenden Gewichten und Bias-Werten gegeben:

Schicht 1: $W_1 = \begin{bmatrix} 2 & 2 \\ 3 & -2 \end{bmatrix}$, $b_1 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$

Schicht 2: $W_2 = \begin{bmatrix} -2 & 2 \end{bmatrix}$, $b_2 = \begin{bmatrix} -1 \end{bmatrix}$

- Stellen Sie dieses Netzwerk graphisch dar. Was ist die Anzahl der Zellen in den einzelnen Schichten?
- Berechnen Sie die Ausgabe für eine Beispiel-Eingabe Ihrer Wahl.
- Stellen Sie ein ein-schichtiges Netz auf, das für jede Eingabe die gleiche Ausgabe wie das obige Netzwerk berechnet und es somit ersetzen könnte.

8.6. NN06: Backpropagation

Videos

- [NN6.1 - MLP Backpropagation 1](#)
- [NN6.2 - MLP Backpropagation 2](#)
- [NN6.3 - MLP Zusammenfassung](#)

Weitere Unterlagen

- [NN06-MLP_Backpropagation.pdf](#)
- [NN06.2-MLP_Backpropagation_Beispiel.pdf](#)

8.6.1. Kurze Übersicht

8.6.1.1. Vorwärts- und Rückwärtslauf

- Im Vorwärtslauf (engl. forward pass oder forward propagation) wird ein einzelner **Vorwärtsschritt** von Schicht $[l - 1]$ auf Schicht $[l]$ wie folgt berechnet:

$$Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]} \quad (1)$$

$$A^{[l]} = g(Z^{[l]}) \quad (2)$$

Dabei bezeichnet g die Aktivierungsfunktion (z.B. Sigmoid oder ReLU).

- Im Rückwärtslauf (engl. *backpropagation*) werden in einem einzelnen **Rückwärtsschritt** von Schicht $[l]$ auf Schicht $[l - 1]$ die folgenden Gradienten berechnet:

$$dZ^{[l]} := \frac{\partial J}{\partial Z^{[l]}} = dA^{[l]} * g'(Z^{[l]}) \quad (3)$$

$$dW^{[l]} := \frac{\partial J}{\partial W^{[l]}} = \frac{1}{m} dZ^{[l]} A^{[l-1]T} \quad (4)$$

$$db^{[l]} := \frac{\partial J}{\partial b^{[l]}} = \frac{1}{m} \sum_{i=1}^m dZ^{[l](i)} \quad (5)$$

$$dA^{[l-1]} := \frac{\partial J}{\partial A^{[l-1]}} = W^{[l]T} dZ^{[l]} \quad (6)$$

Dabei steht “*” für die elementweise Multiplikation.

- Beachten Sie:
 - Der Forwärtsschritt übernimmt $A^{[l-1]}$ von dem vorherigen Schritt und gibt $A^{[l]}$ an den nächsten Schritt weiter.
 - Der Rückwärtsschritt übernimmt $dA^{[l]}$ von dem vorherigen Schritt und gibt $dA^{[l-1]}$ an den nächsten Rückwärtsschritt weiter.

8.6.1.2. Parameteraktualisierung

- Die Aktualisierung der Parameter in Schicht l erfolgt wie gewohnt durch:

$$W^{[l]} = W^{[l]} - \alpha dW^{[l]} \quad (7)$$

$$b^{[l]} = b^{[l]} - \alpha db^{[l]} \quad (8)$$

Dabei bezeichnet α die Lernrate.

Lernziele

- k2: Ich kann den Forwärts- und Rückwärtslauf in Matrix Notation mit mehreren Datenpunkten als Eingabe erklären
- k3: Ich kann Aktivierungsfunktionen ableiten
- k3: Ich kann die Berechnung der partiellen Ableitungen durchführen
- k3: Ich kann den Rückwärtslauf (backpropagation) für ein gegebenes MLP durchführen

Quizzes

- [Selbsttest Backpropagation \(ILIAS\)](#)

8.7. NN07: Training & Testing

Videos

- [NN7.1 - Training, Testing, Validierung](#)
- [NN7.2 - Kreuzvalidierung](#)
- [NN7.3 - Beispiel](#)

Weitere Unterlagen

- [NN07-Testing-Validierung.pdf](#)

8.7.1. Kurze Übersicht

8.7.1.1. Training und Testing

- Der tatsächliche **Erfolg** eines Modells wird nicht durch niedrige Trainingskosten gemessen, sondern durch geringe Kosten auf ungesehenen Daten, d.h. **hohe Vorhersagekraft, gute Generalisierung!**
- Die Menge aller gelabelten Daten in **Trainingsset und Testset** aufteilen, Testset nicht während des Trainings einsetzen!.
 - E_{in} bezeichnet den Fehler auf dem Trainingsset, auch **in-sample error**.
 - E_{out} bezeichnet den Fehler auf dem gesamten Eingaberaum X , auch **out-of-sample error**. E_{out} ist der eigentliche Indikator für den zukünftigen Erfolg des Modells, ist uns aber nicht zugänglich.
 - E_{test} bezeichnet den Fehler auf dem Testset und ist eine **Näherung** für E_{out} .

Tip

Analogie:

E_{in} : Erfolg in Übungsaufgaben und Probeprüfungen.

E_{test} : Erfolg in Endprüfung.

- Die Näherung E_{test} sollte möglichst genau sein, damit es als ein verlässliches **Gütesiegel** dienen kann.
 - Das Testset sollte genug Daten enthalten. Üblicher Anteil an Testdaten:
 - * bei $|D| \approx 100.000 \rightarrow$ ca. 20%
 - * bei $|D| \approx 10.000.000 \rightarrow$ ca. 1%
 - * Beispiel: Hat man 1000 Beispiele im Testset, wird E_{test} mit $\geq 98\%$ Wahrscheinlichkeit in der $\pm 5\%$ Umgebung von E_{out} liegen (für theoretische Grundlagen und Herleitung siehe (Abu-Mostafa u. a. 2012, 39–69)).
 - Trainingsdaten und Testdaten sollten möglichst aus derselben Verteilung kommen, wie die zukünftigen **Real-World-Daten**.
- **Wichtige Bemerkung:**
 - Testdaten nicht anfassen, bis das Modell Einsatzbereit ist!
 - Die Testdaten dürfen in **keinster Weise** bei der Auswahl der endgültigen Hypothese eingesetzt werden, weder bei der Berechnung der Parameter (Training), noch bei der Bestimmung der Hyperparameter (Hyperparameter-Tuning).
 - Sobald der Testfehler die Auswahl der endgültigen Hypothese beeinflusst, kann sie nicht mehr als “Gütesiegel” eingesetzt werden.
CHECK: Hätte man zufällig andere Testdaten gewählt, könnte sich dadurch die endgültige Hypothese ändern?

8.7.1.2. Validierung und Modellauswahl

- Das Ziel ist es, das Modell mit bester Generalisierung, also kleinstem E_{out} zu bestimmen. E_{out} ist jedoch unbekannt und die Näherung E_{test} *darf nicht* bei der Modellauswahl eingesetzt werden.

- **LÖSUNG:** Einen weiteren Teil der Daten als **Validierungsset** (auch *development set*) beiseitelegen und nicht für das Training (i.e. Minimierung des Trainingsfehlers E_{in}) verwenden!

- **Bemerkung:**

Das Wort **Modell** kann je nach Kontext unterschiedliche Bedeutungen annehmen.

Ein Modell im aktuellen Kontext ist als ein Paar $(\mathcal{H}, \mathcal{A})$ von Hypothesenraum (bzw. **Modellarchitektur**) und **Lernalgorithmus** definiert.

- Die Auswahl eines Modells kann aus einer Menge von Modellen unterschiedlicher Art erfolgen (z.B. lineare Modelle, polynomiale Modelle, neuronale Netze), oder von Modellen derselben Art aber mit unterschiedlichen Hyperparametern (z.B. Neuronale Netze mit unterschiedlicher Anzahl von versteckten Schichten).
 - Außerdem kann dieselbe Modellarchitektur $\mathcal{H}$ mit unterschiedlichen Lernalgorithmen trainiert werden, was wiederum die endgültige Hypothese beeinflussen kann. Die Bestimmung der Hyperparameter von $\mathcal{A}$ (wie z.B. Optimierungsfunktion, Lernrate, Kostenfunktion, Regularisierungsparameter usw.) sind daher auch Teil der Modellauswahl.
- Der **Validierungsfehler** E_{val} kann nun als Entscheidungsgrundlage an verschiedenen Stellen des Lernprozesses eingesetzt werden, wie zum Beispiel:
 - Bei der **Auswahl geeigneter Hyperparameter** wie z.B. Anzahl Schichten, Anzahl Zellen/Schicht, Aktivierungsfunktion, Regularisierungsparameter (siehe Abbildung 1).

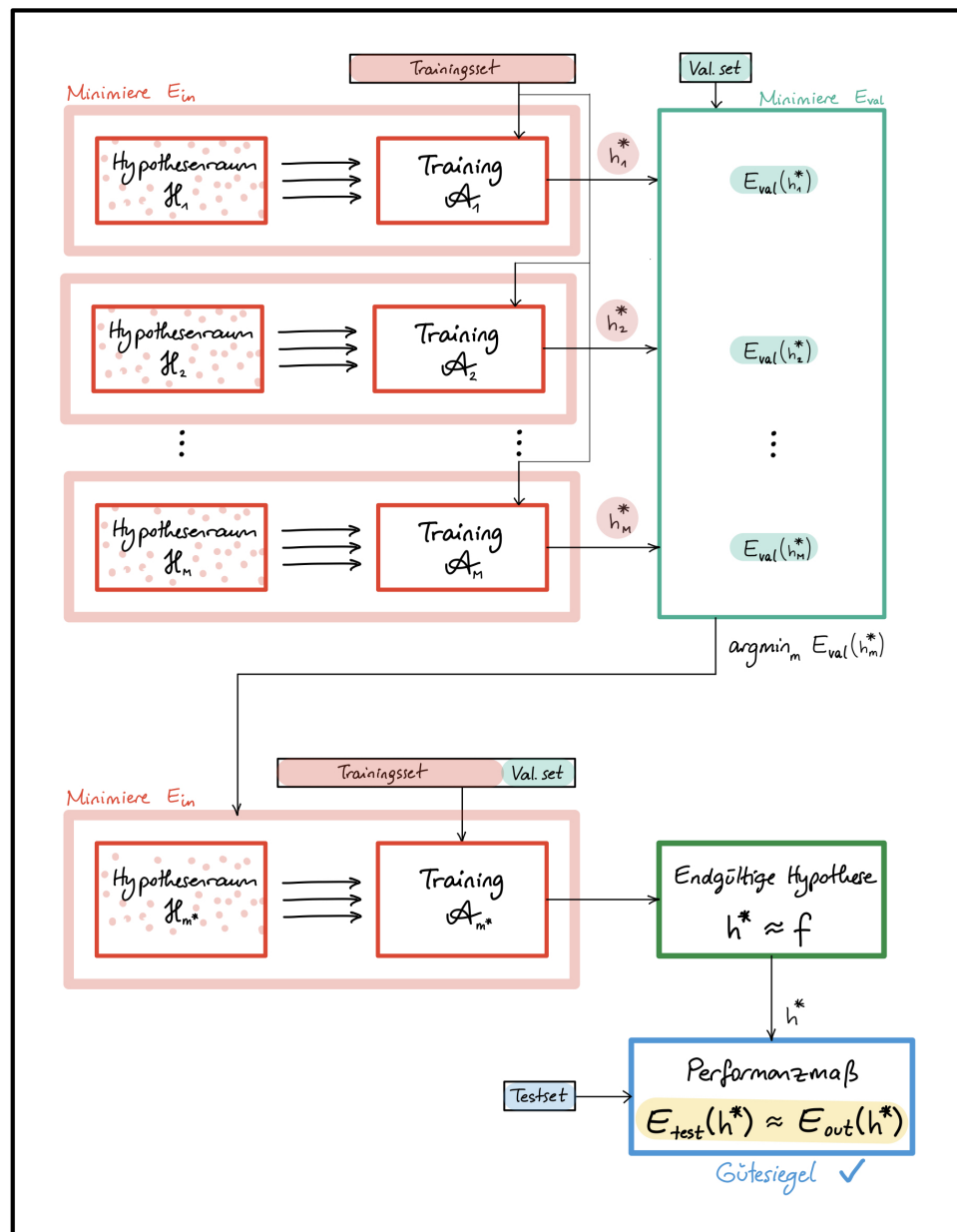


Abbildung 8.3.: Abbildung 1 - Einsatz der Validierung für das Hyperparameter-Tuning

- Bei der **Auswahl der endgültigen Hypothese** ($\rightarrow$ Parameterauswahl!): unter allen Hypothesen, die während des Trainings durchlaufen werden, wähle jene mit kleinstem E_{val} (siehe Abbildung 2).

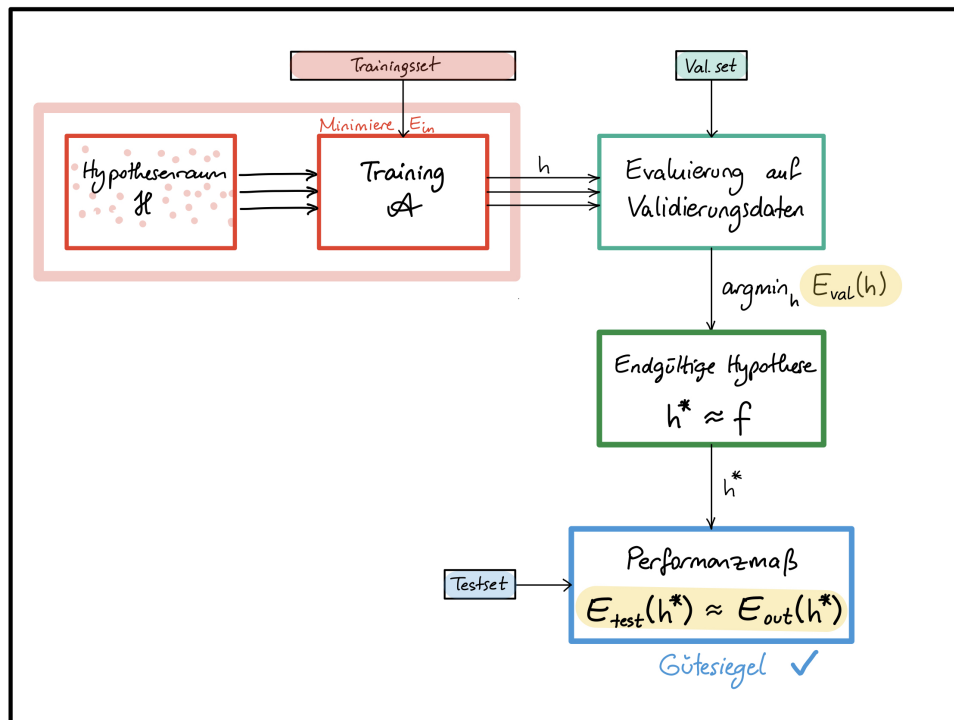


Abbildung 8.4.: Abbildung 2 - Einsatz der Validierung bei der Auswahl der entgültigen Hypothese

- Bei der graphischen **Darstellung von Lernkurven** für die Diagnose von Über- und Unteranpassung (siehe Abbildung 3).

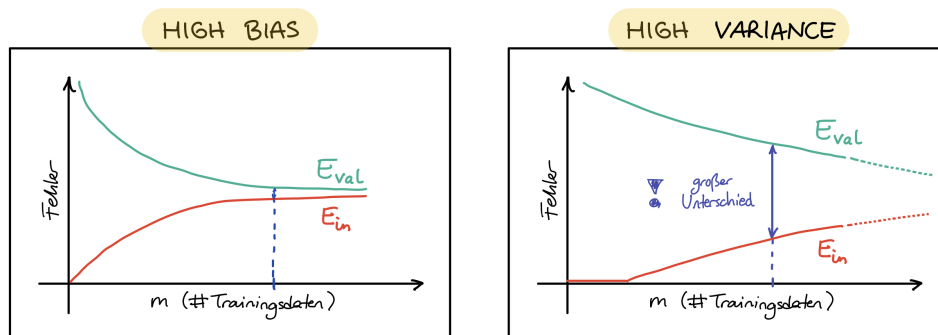


Abbildung 8.5.: Abbildung 3 - Lernkurven

- Übliche train/val/test Aufteilung der Daten (in Prozent):
 - bei $|D| \approx 100.000 \rightarrow \text{ca. } 60/20/20$
 - bei $|D| \approx 10.000.000 \rightarrow \text{ca. } 98/1/1$
- **Bemerkung:**

Das Modell ist trainiert für gute Ergebnisse auf Trainingsdaten und “fine-tuned” für gute Ergebnisse auf den Validierungsdaten. Ergebnisse auf Testdaten werden mit hoher Wahrscheinlichkeit schlechter ausfallen, als auf Validierungsdaten (E_{val} ist eine zu optimistische Näherung).

- Sind Validierungs- und/oder Trainingsset zu klein, führt das zu schlechten Näherungen E_{val} und folglich zu schlechten Entscheidungen.
 - Bei der Aufteilung muss ein gutes Trade-off gefunden werden.
 - Wenn kein Gütesiegel notwendig ist, kann man auf das Testset verzichten und die Daten in Trainings- und Validierungsset aufteilen.
 - Für eine bessere Näherung mit weniger Validierungsdaten kann k-fache Kreuzvalidierung eingesetzt werden (wenn genug Rechenkapazität vorhanden ist).

8.7.1.3. K-fache Kreuzvalidierung (engl. k-fold cross-validation):

- Das Modell $(\mathcal{H}_{\uparrow}, \mathcal{A}_{\uparrow})$ wird k mal trainiert und validiert, jedes mal mit unterschiedlichen Trainings- und Validierungsmengen:
 - Die Trainingsdaten werden in k disjunkte Teilmengen $D_1, D_2, \dots, D_k$ aufgeteilt.
 - Bei dem i -ten Training werden die Teilmenge D_i für die Berechnung des Validierungsfehlers $e_i := E_{val}(h_m^{*(i)})$ und die restlichen $k - 1$ Teilmengen für das Training verwendet.
 - Der **Kreuzvalidierungsfehler** des Modells $(\mathcal{H}_{\uparrow}, \mathcal{A}_{\uparrow})$ ist der Durchschnitt der k Validierungsfehler $e_1, e_2, \dots, e_k$ (siehe Abbildung 4).

$$E_{CV}(m) := \frac{1}{k} \sum_{i=1}^k e_i = \frac{1}{k} \sum_{i=1}^k E_{val}(h_m^{*(i)})$$

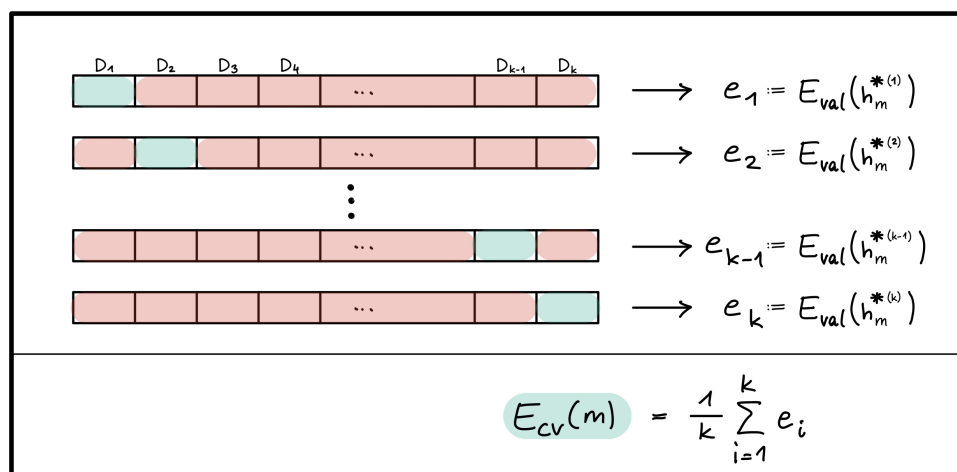


Abbildung 8.6.: Abbildung 4 - Kreuzvalidierung

- Bemerkung: Die Kreuzvalidierung wird nur bei der Modellauswahl eingesetzt: es liefert verlässlichere Näherungen für E_{out} und führt daher zu besseren Entscheidungen. Das zuletzt ausgewählte Modell wird

danach wie gewohnt auf den gesamten Trainingsdaten (ausgenommen Testdaten) trainiert und zum Schluss mit den Testdaten evaluiert.

Lernziele

- k2: Ich kann die Begriffe Trainings-, Validierungs- und Testfehler erklären
- k2: Ich kann den Zweck einer Testmenge erläutern
- k2: Ich kann das Verfahren der Kreuzvalidierung erklären
- k2: Ich kann den Begriff Hyperparameter-Tuning erklären
- k3: Ich kann verschiedene Lernkurven interpretieren

Quizzes

- [Selbsttest Training & Testing \(ILIAS\)](#)

8.8. NN08: Performanzanalyse

Videos

- [NN8.1 - Confusion Matrix](#)
- [NN8.2 - Precision und Recall](#)
- [NN8.3 - Precision Recall Trade-off](#)
- [NN8.4 - F1-Score](#)
- [NN8.5 - Harmonisches Mittel- Intuition](#)

Weitere Unterlagen

- [NN08-Performanzanalyse.pdf](#)

8.8.1. Kurze Übersicht

8.8.1.1. Performanzmetriken für Klassifizierungsprobleme

Wahrheitsmatrix (engl. Confusion Matrix)

- Gibt eine Übersicht über die Anzahl von richtig und falsch klassifizierten Datenpunkten (bei binärer Klassifizierung)
 - TP = # True Positives = Anzahl richtiger 1-Vorhersagen
 - FP = # False Positives = Anzahl falscher 1-Vorhersagen
 - FN = # False Negatives = Anzahl falscher 0-Vorhersagen
 - TN = # True Negatives = Anzahl richtiger 0-Vorhersagen
- Bei Klassifizierungsproblemen mit N Klassen hat man eine $N \times N$ Matrix, die in Position (i, j) die Anzahl der Klasse- j -Beispiele enthält, die als Klasse- i vorhergesagt wurden.

		LABEL (Target)		
		1	0	
VORHERSAGE	1	TP	FP	
	0	FN	TN	TP: Richtig klassifiziert als Positiv FP: Falsch klassifiziert als Positiv TN: Richtig klassifiziert als Negativ FN: Falsch klassifiziert als Negativ

Abbildung 8.7.: Abbildung 1 - Wahrheitsmatrix bei binärer Klassifizierung

Treffergenauigkeit (engl. Accuracy)

- Anzahl richtig klassifizierter Datenpunkte, Erfolgsrate (engl. correct rate)

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

- Accuracy vermittelt ein falsches Bild des Erfolges bei unausgewogenen Datensätzen

Beispiel:

- Klasse 1 hat 10, Klasse 0 hat 990 Beispiele.
- Ein Modell, das immer 0 ausgibt, hat $990/1000 = 0.99$ Treffergenauigkeit, ist aber offensichtlich kein gutes Modell!

Precision

- Positive Predictive Value (PPV)
- Antwort auf: Von allen **positiven Vorhersagen**, wie viele sind richtig?

$$Precision = \frac{TP}{TP + FP}$$

- Wahrscheinlichkeit, dass ein positiv klassifiziertes Beispiel auch tatsächlich positiv ist.
- Je näher an 1, desto besser.
- Accuracy of **positive predictions**.

Recall

- True Positive Rate, auch Sensitivität (engl. Sensitivity)
- Antwort auf: Von allen **positiven Beispielen**, wie viele wurden richtig klassifiziert?

$$Recall = \frac{TP}{TP + FN}$$

- Wahrscheinlichkeit, dass ein positives Beispiel tatsächlich als solches erkannt wird.
- Je näher an 1, desto besser.
- Accuracy of **positive examples**.

Precision-Recall Trade-off

- Ein gutes Modell sollte hohe Precision und zugleich hohes Recall haben.
- Man kann die Precision eines Modells beliebig erhöhen (durch das Vergrößern des Schwellenwertes bei der Klassifizierung), jedoch wird dabei der Recall abnehmen.
- Genau so kann man den Recall eines Modells beliebig erhöhen (durch das Verkleinern des Schwellenwertes bei der Klassifizierung), jedoch wird dabei die Precision abnehmen.
- Es gilt ein gutes Trade-off zu finden.
- Eine Zwei-Zahlen-Metrik erschwert den Entscheidungsprozess bei Evaluierung und Modellauswahl.

F_1 -Score (Harmonisches Mittel)

- Fasst Precision (P) und Recall (R) in einer Metrik zusammen (Harmonisches Mittel von P und R):

$$F_1 - Score = \frac{2}{\frac{1}{P} + \frac{1}{R}} = 2 \cdot \frac{PR}{P + R}$$

- Der F_1 -Score wird nur dann hoch sein, wenn P und R beide hoch sind.
- Je näher an 1, desto besser.
- Sehr kleine P und R Werte ziehen den F_1 -Score sehr stark herunter. In dieser Hinsicht gibt diese Metrik ein akkurates Bild über den Erfolg eines Modells.

Lernziele

- k2: Ich kann die betrachteten Performanzmetriken für die Evaluierung von Klassifizierungsmodellen erläutern
- k2: Ich kann die Wahrheitsmatrix (engl. Confusion Matrix) erklären
- k2: Ich kann den Begriff Treffergenauigkeit (engl. Accuracy) erklären
- k2: Ich kann den Begriff Precision (engl. Precision) erklären
- k2: Ich kann den Begriff Recall erklären
- k2: Ich kann den Begriff F_1 -Score (Harmonisches Mittel) erklären
- k3: Ich kann Precision und Recall berechnen und deuten
- k3: Ich kann den F_1 -Score berechnen und deuten
- k3: Ich kann die Performanzmetriken bei der Evaluierung und Auswahl von Modellen einsetzen

9. Naive Bayes

Ich habe Symptome beobachtet. Kann ich die Ursache (also die Krankheit) bestimmen, wenn ich etwas Hintergrundwissen habe:

- Wie häufig treten verschieden Krankheiten auf
- Welche Krankheit zeigt welche Symptome (und wie oft treten die dann auf)

Kann ich aus diesen Daten einen Klassifikator lernen?

Teil III.

Organisatorisches

10. Prüfungsvorbereitung (HSBI)

TL;DR

Durchführung: Präsenz

Die Klausur wird dieses Semester elektronisch stattfinden. Dazu werden wir den E-Assessment-ILIAS der HSBI nutzen.

Die Klausur wird in den Räumen der HSBI am Campus Minden unter Aufsicht durchgeführt. Hier werden Ihnen Rechner für den Zugang zum Prüfungs-ILIAS zur Verfügung gestellt, Sie benötigen nur Ihre HSBI-Zugangsdaten (Username, Passwort), einen Studierendenausweis und Personalausweis sowie Ihren DIN-A4-Spickzettel.

Ablauf der Klausur

Die Prüfung (das ILIAS-Objekt) selbst schalte ich zum Start der Prüfung online. Sie loggen sich vor Ort in den ILIAS mit Ihren HSBI-Zugangsdaten ein und können die Prüfung nach dem gemeinsamen Start beginnen. Nach Ablauf der Bearbeitungszeit beenden Sie die Prüfung und loggen sich wieder aus. Parallel zur Prüfung werde ich in einer Zoom-Sitzung zur Verfügung stehen, in der Sie Fragen stellen können.

Hilfsmittel und Themen

Als Hilfsmittel ist ein Spickzettel (DIN A4, beidseitig beschrieben) zugelassen.

Es wird keines der behandelten Themen ausgeschlossen, allerdings eignen sich manche Themen etwas besser für Klausurfragen als andere ...

Videos

1. Fragetypen-Demo
 - **Video** zur Fragetypen-Demo: [\[YT\]](#), [\[HSBI\]](#)
 - **Fragetypen-Demo** im ILIAS
2. Hinweise zur Prüfung: [\[YT\]](#), [\[HSBI\]](#)

10.1. Elektronische Klausur: Termin, Materialien

10.1.1. Termin

Die schriftliche Prüfung erfolgt durch eine Klausur, die als digitale Prüfung auf einem Prüfungs-ILIAS durchgeführt wird.

Die Klausur wird in Präsenz in den Rechnerpools am Campus Minden durchgeführt.

Es wird in beiden Prüfungszeiträumen ein Termin angeboten. Die Termine werden vom Prüfungsamt bekannt gegeben.

Dauer: siehe Aushang bzw. Mail.

10.1.2. Zugelassene Hilfsmittel: DIN-A4-Spickzettel (beidseitig)

Sie dürfen **einen** Spickzettel im **DIN-A4**-Format benutzen, der beidseitig beschrieben sein kann.

Ich möchte Sie hier noch einmal ermuntern, diesen Zettel tatsächlich manuell zu erstellen (also ganz traditionell zu **schreiben**), da bereits der Schreibvorgang einen gewissen Lerneffekt bewirkt! (Wenn Sie sich bewusst dafür entscheiden, können Sie den Spickzettel natürlich auch ausdrucken.)

10.1.3. Einsicht

Die Prüfungseinsicht findet möglichst zeitnah und online statt. Details werden per Mail bekanntgegeben.

10.2. Technische Vorbereitungen

Sie benötigen am Prüfungstag:

1. **HSBI-Zugangsdaten:** Username, Passwort

Bei der Durchführung der Prüfung am Campus Minden wird Ihnen ein Rechner zur Verfügung gestellt. Dort läuft voraussichtlich ein Browser im Kiosk-Mode, wo Sie sich am Prüfungs-ILIAS anmelden. Dazu benötigen Sie Ihre HSBI-Zugangsdaten, mit denen Sie sich auch im "normalen" ILIAS anmelden.

2. **Studierendenausweis** und Personalausweis

An der Prüfung dürfen nur Personen teilnehmen, die dafür im LSF/CAT angemeldet sind. Es findet eine entsprechende Kontrolle statt. Halten Sie dazu Ihren Studierendenausweis und Personalausweis bereit.

10.3. Bearbeitung des E-Assessment

1. Lesen Sie sich die Hinweise auf der Startseite durch
2. Bearbeiten Sie die Aufgaben in **einem einzigen** Browser-Tab

Öffnen Sie die Aufgaben NICHT in parallelen Tabs! Es kann sonst zu Fehlfunktionen von ILIAS kommen.

Bewegen Sie sich **nicht per Browser-Navigation** ("vor", "zurück" im Browser) durch die Aufgaben, sondern nutzen Sie dafür die **Buttons "nächste Frage", "Weiter" oder "Zurück" in ILIAS!**

3. Hinweis zur Anzeige der restlichen Bearbeitungsdauer

Wenn Sie den Browser bzw. das Tab mit der Prüfung im Laufe der Prüfung verlassen, wird Ihnen bei der Rückkehr unter Umständen eine falsche restliche Bearbeitungsdauer angezeigt. Sie können die Anzeige korrigieren/aktualisieren, indem Sie einfach zu einer vorigen oder nächsten Aufgabe navigieren.

Hinweis: Die restliche Bearbeitungsdauer wird im Test nur dann angezeigt, wenn diese Funktion von den Prüfenden aktiviert wurde.

4. Parallel zum E-Assessment läuft eine Zoom-Session, dort können Sie Fragen stellen

10.4. Fragetypen-Demo

In Ihrem ILIAS-Kurs finden Sie eine [Fragetypen-Demo](#) mit den wichtigsten Fragetypen. Machen Sie sich mit der Mechanik der Fragetypen vertraut und schauen Sie sich die Kommentare bei den einzelnen Aufgaben an. Sie können die Demo bei Bedarf beliebig oft wiederholen.

10.5. Hinweise zu den Inhalten

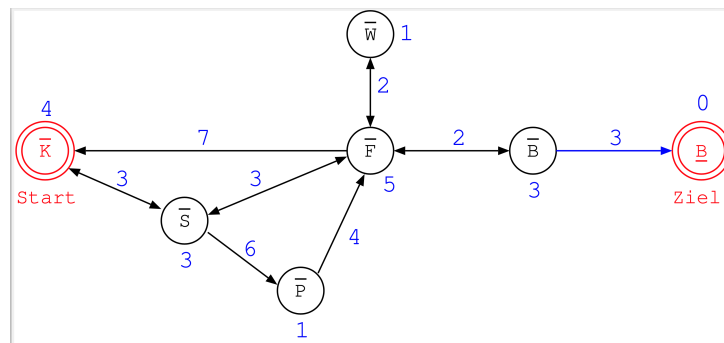
- Klausurrelevant: Vorlesung und Praktikum
- Für Verständnis u.U. hilfreich: Studium der vertiefenden Literaturangaben
- **Fragen:**
 - Schauen Sie sich die Challenges an ...
 - Schauen Sie sich die Praktikumsaufgaben an ...
 - Überlegen Sie sich, was zu einem Themengebiet im Rahmen einer Prüfung möglich ist und (wie) gefragt werden könnte ...

Können vor Kennen

10.6. Beispiele für mögliche Fragen

10.6.1. Breitensuche

Betrachten Sie den folgenden Graphen:



Mittels Breitensuche (Variante “Tree-Search”) soll der kürzeste Weg von “K” nach “B” gefunden werden. Die folgende Queue stellt den aktuellen Zustand der Suche dar:

[KSF, KSP]

Führen Sie den nächsten Schritt des Breitensuche-Algorithmus durch und geben Sie die resultierende Queue an. Achten Sie auf die korrekte Sortierung und halten Sie sich an die vorgegebene Notation.

Hinweis: Die vorgegebene Queue kann willkürlich konstruiert sein und muss nicht viel mit Breitensuche zu tun haben.

10.6.2. Suche und Constraints

- Worin liegt der Unterschied zwischen einem Knoten und einem Zustand?
- Was ist ein Suchbaum? Wo liegt der Unterschied zum Zustandsraum?
- Muss der Suchbaum über eine eigene Datenstruktur realisiert werden?
- Was bedeutet Kantenkonsistenz?

Erklären Sie die jeweiligen Begriffe nachvollziehbar. Nutzen Sie dazu ein eigenes kleines Beispiel.

10.6.3. Handsimulation Perzeptron

Gegeben sei folgendes Perzeptron mit den Gewichten $w_0 = 0$, $w_1 = 0$, $w_2 = 0$. Betrachten Sie den folgenden Eingabevektor $x_0 = 1$, $x_1 = 1$, $x_2 = 0$ mit der Klasse $+1$. Geben Sie für diesen Vektor folgende Werte an:

- Skalarprodukt $\mathbf{w}^T \mathbf{x}$,
- Aktivierung $\text{sgn}(\mathbf{w}^T \mathbf{x})$.
- Falls notwendig, führen Sie einen Lernschritt durch. Nutzen Sie dabei $\alpha = 1$.

Geben Sie die resultierenden (neuen) Gewichte an (auch wenn Sie keinen Lernschritt durchführen mussten).

10.6.4. Entscheidungsbäume mit ID3

Gegeben sei folgender Trainingsdatensatz ...

Mit Hilfe von ID3 soll für diesen Datensatz ein Entscheidungsbaum konstruiert werden. Der Algorithmus läuft schon und hat das Merkmal x_2 als Wurzelknoten ausgewählt. Beschreiben Sie detailliert, wie ID3 die nächste Ebene aufbaut. Gehen Sie dabei konkret auch auf den Algorithmus ein. Geben Sie die eingesetzten Formeln für Entropien und Gains an und erklären Sie Ihr Vorgehen.

Hinweis: Sie sollen die Formeln nur einsetzen, aber nicht ausrechnen.

10.6.5. CSP

Betrachten Sie das folgende Constraintproblem:

- $C_1 = \{(a, b) | a \neq b\}$
- $C_2 = \{(b, c) | b \neq c\}$
- $C_3 = \{(a, c) | a = c\}$

- $D_a = D_b = D_c = \{1, 2, 3, 4, 5\}, D_d = \{1, 2, 5\}, D_e = \{1, 3\}$

Zeigen Sie per Handsimulation, wie der Algorithmus AC-3 schrittweise Kantenkonsistenz herstellt. Hier ist die Queue in einem bestimmten Schritt: ... Führen Sie einen Schritt durch und geben Sie die resultierende Queue an. Erklären Sie Ihr Ergebnis.

10.6.6. NB

Betrachten Sie den folgenden Datensatz: ... “Trainieren” Sie für diesen Datensatz einen vollständigen Naive-Bayes-Klassifikator. Nutzen Sie die Variante “Multinomial NB”. Geben Sie alle Bestandteile an. Erklären Sie Ihr Vorgehen nachvollziehbar.

Hinweis: Sie brauchen die Brüche nicht ausrechnen.

Quellen

Abu-Mostafa, Y. S., M. Magdon-Ismael, und H. Lin. 2012. *Learning From Data*. AMLBook. <https://work.caltech.edu/telecourse>.

Russell, S., und P. Norvig. 2021. *Artificial Intelligence: A Modern Approach*. 4th Edition. Pearson. <http://aima.cs.berkeley.edu>.



Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Exceptions:

- “[künstliche intelligenz](#)” by [Gerd Altmann \(geralt\)](#) on Pixabay.com ([Pixabay License](#))
- “[Exp e.svg](#)” by Marcel Marnitz, reworked by [Georg-Johann](#) on Wikimedia Commons ([Public Domain](#))
- [MapGermanyGraph.svg](#) by [Regnaron](#) and [Jahobr](#) on Wikimedia Commons ([Public Domain](#))

Last modified: bc5bcf0 2026-09-04 add inverted images